

Università degli Studi di Napoli Federico II
Corso di Ingegneria del Software

BugBoard26

Documento dei Requisiti Software

Gruppo: INGSW2526_048
Matricole: N86004715, N86004797

21 ottobre 2026

Indice

I Documento dei Requisiti Software **4**

1 Glossario **5**

2 Introduzione **7**

3 Requirement Elicitation and Analysis **9**
3.1 Individuazione del target degli utenti tramite Personas 9

4 Requirement Specification **13**
4.1 Use Case Diagram 13
4.2 Requisiti Funzionali 14
4.3 Requisiti Non Funzionali 16
4.4 Requisiti di Dominio 17
4.5 Rappresentazione Tabellare Modello di A.Cockburn 19

II Documento di Design del Sistema **26**

5 Progettazione Architetturale e Scelte Tecnologiche **27**
5.1 Descrizione dell'Architettura 27
5.1.1 Controller 27
5.1.2 Service 27
5.1.3 Model e accesso ai dati 27
5.2 Architettura del Frontend 28
5.3 Criteri di Design e Motivazioni 29
5.4 Tecnologie Utilizzate 30
5.5 Schema della Persistenza dei Dati 31
5.5.1 Descrizione delle entità e delle relazioni 31

6 Design dell'interfaccia utente **33**
6.1 Progettazione Cromatica dell'Interfaccia 33
6.1.1 Obiettivi della Progettazione Cromatica 33
6.1.2 Palette Cromatica Adottata 34
6.1.3 Applicazione della Regola 60–30–10 34
6.1.4 Accessibilità e Contrasto Cromatico 35
6.2 Tipografia 36
6.2.1 Obiettivi della Progettazione Tipografica 36
6.2.2 Scelta del Font 36
6.2.3 Gerarchia Tipografica 37

6.2.4 Spaziatura e Allineamento . . . 37
6.2.5 Accessibilità e Conformità alle WCAG . . . 37
6.2.6 Coerenza con l'Identità Visiva . . . 37
6.3 Principles of Proximity and Connectedness . . . 38
6.4 Applicazione del modello F-Pattern . . . 42
6.5 Labor Perception Bias . . . 43
6.6 Gestione del Feedback Utente e Feedback Persistence . . . 43
6.7 Prevenire gli Errori . . . 44

7 Diagrammi delle classi di Design 47
7.1 Classi di Design del Backend . . . 47
7.1.1 Architettura Controller–Service–Model . . . 47
7.1.2 DTO . . . 47
7.1.3 Entità di Dominio e sicurezza . . . 48
7.2 Classi di Design del Frontend . . . 49
7.2.1 Utility, Config e Security . . . 51
7.3 Gestione della navigazione tra le pagine . . . 53
7.4 Sequenze di Design per gli Use Case . . . 56
7.4.1 UC1 – Creazione issue . . . 56
7.4.2 UC2 – Cambio stato . . . 56
7.4.3 UC3 – Commento . . . 56
7.4.4 UC4 – Creazione account e appartenenza . . . 56

III Documentazione del Processo di Sviluppo 57

8 File di build automatica 58
8.1 DockerFile (Backend) . . . 58
8.2 DockerFile (Frontend) . . . 58
8.3 Docker Compose . . . 58

9 Strumento di Versioning 60

10 Qualità del codice 61
10.1 Grafico di Duplicazione del Codice . . . 61
10.2 Grafico delle Issues . . . 62
10.3 Grafico della Coverage dei Test . . . 62

IV Testing e Valutazione dell'Usabilità 63

11 Unit Testing 64
11.1 Test di createIssue (issues service) . . . 64
11.2 Test di changeStatus (issues service) . . . 65
11.3 Test di requireAssignee (access service) . . . 66
11.4 Test di notify (notifications service) . . . 66
11.5 Strategie adottate . . . 67
11.5.1 Controller . . . 67
11.5.2 Service . . . 67

11.5.3 Model / persistenza 67
11.5.4 Security e Utils 67
11.6 Test plan di una funzionalità: risoluzione issue 68

12 Valutazione dell'Usabilità **69**
12.1 Expert Review 69
12.1.1 Obiettivo del Test 69
12.1.2 Istruzioni per l'Uso della Checklist 69
12.1.3 Risultati 69
12.2 Relazione Sperimentale 70
12.2.1 Descrizione dei Partecipanti 70
12.2.2 Procedura Sperimentale 70
12.2.3 Dati Raccolti 71
12.2.4 Valutazione post-task 71
12.2.5 Conclusioni 71

Parte I

Documento dei Requisiti Software

Capitolo 1

Glossario

1. **Utente**
   Persona registrata che accede alla piattaforma BugBoard26 e può segnalare issue, visualizzarle e interagire con esse secondo i permessi assegnati.

2. **Amministratore**
   Utente con accesso globale ai progetti: può creare account e progetti, aggiungere e rimuovere membri e operare sulle issue. L'eliminazione dei progetti, pur presente nei mockup iniziali, non è implementata nello snapshot.

3. **Progetto**
   Contesto di lavoro che raccoglie un insieme di issue correlate. Ogni progetto può includere più utenti e contiene le segnalazioni ad esso associate.

4. **Appartenenza a Progetto**
   Associazione tra un utente e un progetto, che determina la possibilità per l'utente di accedere e operare all'interno di quel progetto.

5. **Issue**
   Segnalazione relativa a un progetto software, identificata almeno da un titolo e una descrizione, che rappresenta un problema, una richiesta o un suggerimento.

6. **Bug**
   Tipo specifico di issue che segnala un malfunzionamento del sistema e che può essere assegnato a un membro del team per la risoluzione.

7. **Question**
   Tipo di issue utilizzato per richieste di chiarimento o domande relative al progetto.

8. **Documentation**
   Tipo di issue utilizzato per segnalare problemi o miglioramenti relativi alla documentazione del progetto.

9. **Feature**
   Tipo di issue utilizzato per proporre nuove funzionalità o miglioramenti del sistema.

10. **Commento**
    Messaggio associato a una issue, inserito dagli utenti per fornire aggiornamenti, chiarimenti o osservazioni.

11. **Priorità**
    Livello facoltativo associato a una issue: `LOW`, `MEDIUM`, `HIGH` o `CRITICAL`; se omesso rimane `null`, visualizzato come *Not specified*.

12. **Stato**
    Condizione corrente di una issue nel suo ciclo di vita, ad esempio `TODO`, `IN_PROGRESS` o `CLOSED`.

13. **Notifica**
    Comunicazione generata dal sistema quando avvengono eventi rilevanti, come l'assegnazione di un bug o la sua risoluzione.

14. **API REST**
    Interfaccia di comunicazione tra frontend e backend basata sul protocollo HTTP e sul formato di scambio dati JSON.

15. **DTO**
    Strutture JSON di richiesta e risposta, costruite da funzioni di serializzazione; non classi DTO Java. I dati pubblici non espongono gli hash delle password.

16. **Token**
    Stringa generata dal sistema per autenticare un utente e autorizzare l'accesso alle risorse protette.

Capitolo 2

Introduzione

L'obiettivo di questo documento è definire formalmente i requisiti funzionali, i requisiti non funzionali e i vincoli di dominio del sistema **BugBoard26**, una piattaforma software progettata per supportare la gestione collaborativa delle issue all'interno di progetti software.

Il sistema consente ai membri di un team di sviluppo di segnalare problemi, monitorarne lo stato e collaborare alla loro risoluzione. Attraverso l'applicazione, gli utenti autenticati possono creare nuove issue, visualizzare l'elenco delle segnalazioni presenti nel sistema, aggiungere commenti e aggiornare lo stato dei bug assegnati.

Il progetto è sviluppato nell'ambito del corso di Ingegneria del Software dell'Università degli Studi di Napoli "Federico II" e segue un approccio orientato ai requisiti, che rappresenta la fase iniziale del processo di sviluppo software e precede le successive attività di progettazione architetturale, implementazione e testing.

A seguito del calcolo previsto nella traccia del progetto, al nostro gruppo sono state assegnate le seguenti funzionalità obbligatorie del sistema:

- Autenticazione degli utenti tramite email e password.
- Creazione di nuove issue da parte degli utenti autenticati.
- Visualizzazione e consultazione dell'elenco delle issue presenti nel sistema, con possibilità di organizzazione e filtraggio.
- Inserimento di commenti associati alle issue per favorire la collaborazione tra i membri del team.
- Aggiornamento dello stato di un bug da parte dell'utente a cui esso è stato assegnato.

Le funzionalità sopra elencate rappresentano il nucleo principale del sistema BugBoard26 e consentono di gestire in modo efficace il processo di segnalazione, discussione e risoluzione dei problemi all'interno di un progetto software.

- **1 – Autenticazione utenti.** Il sistema deve consentire agli utenti di accedere alla piattaforma tramite autenticazione basata su email e password. L'autenticazione deve garantire la sicurezza delle informazioni gestite dal sistema e permettere l'identificazione univoca degli utenti che interagiscono con la piattaforma. Un amministratore dispone di privilegi avanzati e può creare nuovi account specificando email, password e ruolo dell'utente.

- **2 – Creazione di nuove issue.** Gli utenti autenticati devono poter segnalare nuove issue all'interno del sistema. Ogni issue deve essere caratterizzata almeno da un titolo e da una descrizione testuale che descriva il problema o la richiesta. Le issue possono appartenere a diverse categorie, come *bug*, *feature*, *documentation* o *question*. La priorità è facoltativa. L'immagine è un miglioramento previsto nel progetto iniziale, ma il caricamento di allegati non è implementato. Alla creazione, ogni issue viene inizialmente impostata nello stato `TODO`.

- **3 – Visualizzazione e consultazione delle issue.** Il sistema deve fornire una vista riepilogativa delle issue presenti, permettendo agli utenti di consultare le segnalazioni registrate. L'interfaccia deve consentire l'organizzazione delle issue tramite criteri di ordinamento o filtraggio, ad esempio per tipologia o stato, in modo da facilitare la consultazione e la gestione delle segnalazioni.

- **5 – Gestione dei commenti.** Ogni issue deve includere una sezione dedicata ai commenti, nella quale gli utenti possono inserire messaggi, aggiornamenti o richieste di chiarimento. Questa funzionalità favorisce la collaborazione tra i membri del team e consente di mantenere traccia delle discussioni relative alla risoluzione di un problema.

- **6 – Aggiornamento dello stato di un bug.** Un bug assegnato a un utente può essere aggiornato nel suo stato durante il processo di risoluzione. L'utente responsabile può modificare lo stato del bug per riflettere l'avanzamento delle attività (`TODO`, `IN_PROGRESS`, `CLOSED`). Quando un bug raggiunge lo stato `CLOSED`, il sistema deve notificare l'utente che ha originariamente segnalato il problema.

Queste funzionalità rappresentano il nucleo principale del sistema BugBoard26 e costituiscono la base per le successive attività di progettazione architetturale, implementazione e testing del software.

Stato dell'implementazione e ambito della relazione La descrizione è riferita ai sorgenti forniti il 21 ottobre 2026: frontend React/Vite, backend Node.js/Express, ORM Sequelize e PostgreSQL. Gli USER operano nei progetti di appartenenza; gli ADMIN hanno accesso globale. Il cambio stato non è riservato esclusivamente all'assegnatario. Il confronto della limitazione per membership con la traccia ufficiale rimane una decisione da confermare. La presenza di codice, gli esiti di test dichiarati nelle note e le verifiche effettivamente eseguite sono distinti nei capitoli su qualità, testing e usabilità.

Capitolo 3

Requirement Elicitation and Analysis

3.1 Individuazione del target degli utenti tramite Personas

Gli utenti principali del sistema **BugBoard26** appartengono a due categorie principali: utenti del team di sviluppo e amministratori del sistema. La piattaforma è progettata per supportare la gestione collaborativa delle issue all'interno di un progetto software, permettendo ai membri del team di segnalare problemi, discuterli e monitorarne lo stato di risoluzione.

Attraverso l'utilizzo delle *personas* vengono rappresentati archetipi di utenti reali, con l'obiettivo di comprendere meglio i loro bisogni, le loro aspettative e il contesto di utilizzo del sistema. Questa tecnica consente di progettare le funzionalità della piattaforma in modo più mirato, garantendo un'interazione semplice ed efficace.

Nel contesto di BugBoard26 sono state individuate le seguenti tipologie di utenti:

- **Utente del sistema:** È un membro del team di sviluppo che utilizza la piattaforma per segnalare problemi o richieste relative al progetto software. Può creare nuove issue, consultare le segnalazioni esistenti, aggiungere commenti e collaborare alla risoluzione dei bug.
- **Utente assegnatario del bug:** È un utente del sistema a cui viene assegnata la responsabilità di risolvere un determinato bug. Questo utente può aggiornare lo stato dell'issue per riflettere l'avanzamento del lavoro fino alla sua risoluzione.
- **Amministratore:** È l'utente con privilegi più elevati all'interno della piattaforma. L'amministratore ha il compito di gestire gli account degli utenti e può creare nuovi utenti specificando le relative credenziali e i privilegi di accesso al sistema.

Persona 1 – Giulia Bianchi



Figura 3.1: Persona 1 – Giulia Bianchi

Persona 2 – Marco Ferri



Figura 3.2: Persona 2 – Marco Ferri

Persona 3 – Luca Lauro



Figura 3.3: Persona 3 – Luca Lauro

Capitolo 4

Requirement Specification

4.1 Use Case Diagram

In questa sezione viene presentato il diagramma dei casi d'uso del sistema **BugBoard26**. Il diagramma rappresenta gli attori che interagiscono con il sistema e le principali funzionalità offerte dalla piattaforma per la gestione collaborativa delle issue.

Gli attori principali individuati sono:

- **Utente**: può creare issue, visualizzare le segnalazioni presenti nel sistema, filtrare i risultati, commentare e consultare le notifiche.
- **Utente assegnatario**: ruolo nel contesto dell'issue; può aggiornare lo stato se autorizzato, ma non è l'unico utente ammesso alla modifica.
- **Admin**: specializzazione dell'utente base che possiede privilegi amministrativi, come creare account e progetti, aggiungere o rimuovere membri e aggiornare le issue con accesso globale. L'eliminazione del progetto non è implementata.

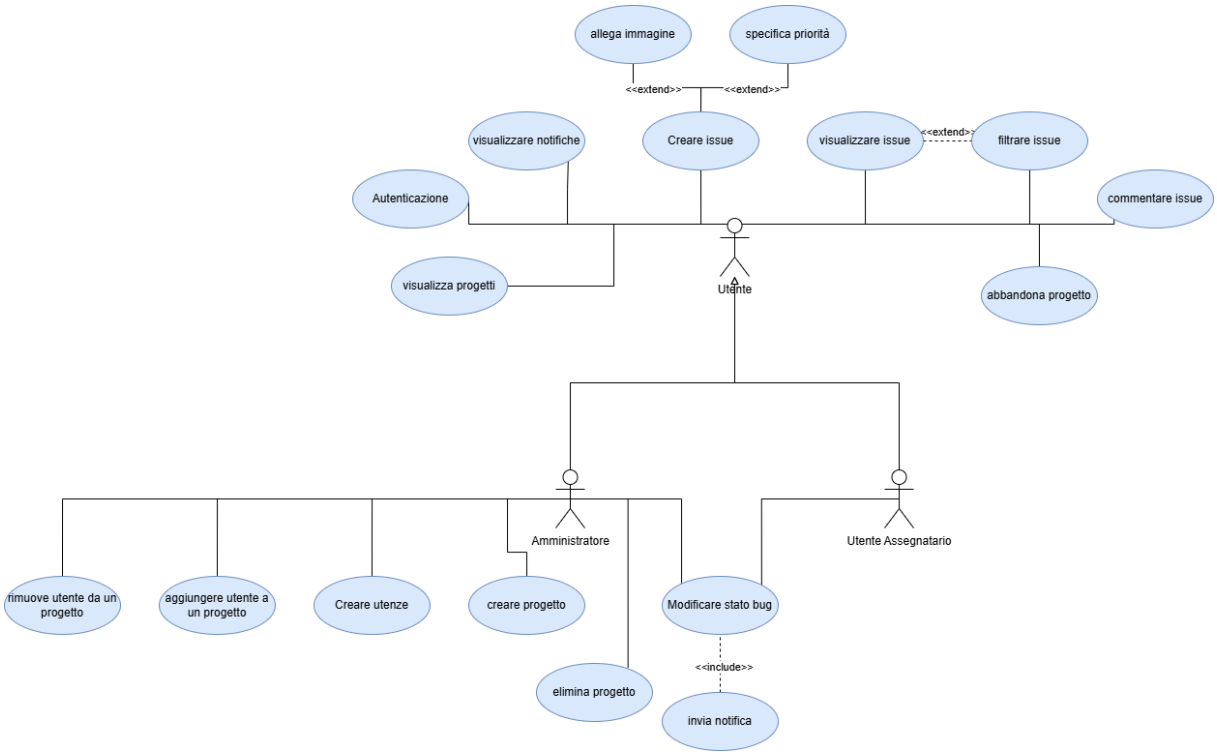


Figura 4.1: Use Case Diagram originale di progetto. Le funzionalità previste ma non implementate e i permessi effettivi sono distinti nel testo e nei requisiti seguenti.

4.2 Requisiti Funzionali

I requisiti funzionali descrivono i servizi che il sistema BugBoard26 mette a disposizione degli utenti.

ID	DENOMINAZIONE	DESCRIZIONE
RF1	Login Utente	Un utente registrato può accedere alla piattaforma inserendo email e password valide.
RF2	Logout Utente	Il logout elimina la sessione nel browser; il JWT non è revocato immediatamente sul server.
RF3	Validazione Credenziali	Il sistema verifica la correttezza delle credenziali fornite in fase di autenticazione prima di consentire l'accesso.
RF4	Creazione Utente (Admin)	ADMIN crea un account con nome completo, email, password e ruolo USER o ADMIN.

ID	DENOMINAZIONE	DESCRIZIONE
RF5	Gestione Ruolo Utente	Il sistema distingue gli utenti in base al ruolo assegnato, abilitando funzionalità differenti per amministratori e utenti normali.
RF6	Creazione Issue	Un membro del progetto o un ADMIN può creare una issue in un progetto accessibile.
RF7	Inserimento Titolo e Descrizione	Durante la creazione di una issue, l'utente deve inserire almeno un titolo e una descrizione testuale.
RF8	Classificazione Issue	Il tipo è obbligatorio: BUG, FEATURE, DOCUMENTATION o QUESTION.
RF9	Stato Iniziale Issue	Il server imposta lo stato iniziale a `TODO`, indipendentemente da valori inviati dal client.
RF10	Impostazione Priorità	La priorità è facoltativa; se omessa resta `null`, senza un livello predefinito.
RF11	Allegato Immagine (previsto)	Miglioramento facoltativo del progetto iniziale; upload e storage immagini non sono implementati.
RF12	Visualizzazione Elenco Issue	Un utente può visualizzare l'elenco delle issue relative a un progetto selezionato.
RF13	Visualizzazione Dettaglio Issue	Un utente può consultare il dettaglio di una specifica issue, comprensivo delle informazioni associate.
RF14	Filtraggio Issue	Il sistema consente di filtrare le issue di un progetto in base a criteri rilevanti, come tipologia, stato o priorità.
RF15	Ordinamento Issue	Il backend ordina per `updatedAt` e ID decrescenti. Non è disponibile un criterio selezionabile; i filtri sono implementati.
RF16	Visualizzazione Commenti	Un utente può visualizzare i commenti associati a una issue.
RF17	Inserimento Commento	Un utente autenticato può inserire un commento all'interno di una issue per fornire aggiornamenti o chiarimenti.
RF18	Aggiornamento Stato Issue	Membro del progetto o ADMIN può eseguire `TODO` → `IN_PROGRESS` → `CLOSED`; il codice applica le transizioni a tutte le tipologie.

ID	DENOMINAZIONE	DESCRIZIONE
RF19	Controllo di Accesso alla Modifica	Il vincolo attuale è l'accesso al progetto: non è richiesta l'identità esclusiva dell'assegnatario.
RF20	Notifica Risoluzione	La chiusura genera una notifica RESOLUTION al reporter, anche se è l'attore o un ex membro del progetto.
RF21	Notifiche Personali	Il destinatario legge e marca le proprie notifiche; la risoluzione resta leggibile al reporter ex membro con `issueId=null` nella risposta, senza riacquisire accesso all'issue.
RF22	Visualizzazione Progetti	USER vede i propri progetti; ADMIN può accedere a tutti i progetti.
RF23	Selezione Progetto	Un utente può selezionare un progetto per accedere alle issue associate e gestirle.
RF24	Aggiunta Membro	ADMIN seleziona un account esistente per email nell'interfaccia; l'API riceve userId.
RF25	Abbandono Progetto (previsto)	Funzionalità presente nel progetto iniziale ma non implementata nello snapshot.
RF26	Un amministratore può rimuovere un utente da un progetto	Un amministratore può rimuovere un utente da un progetto, eliminando la relativa associazione.
RF27	Creazione Progetto	Un amministratore può creare un nuovo progetto, specificandone almeno nome e descrizione.
RF28	Eliminazione Progetto (prevista)	Funzionalità presente nel progetto iniziale ma non implementata nello snapshot.

4.3 Requisiti Non Funzionali

I requisiti non funzionali definiscono i vincoli e le caratteristiche qualitative che il sistema BugBoard26 deve rispettare per garantire sicurezza, affidabilità e qualità del servizio.

DENOMINAZIONE	DESCRIZIONE
Unicità Email	Nel sistema può esistere un solo account associato a una determinata email, al fine di evitare duplicazioni e garantire l'univocità degli utenti.

DENOMINAZIONE	DESCRIZIONE
Unicità Associazione Utente-Progetto	Nel sistema non possono esistere due associazioni identiche tra lo stesso utente e lo stesso progetto, al fine di evitare duplicazioni nella tabella *PROJECT_MEMBERSHIP*.
Sicurezza delle Credenziali	Il sistema deve garantire una gestione sicura delle credenziali degli utenti, proteggendo password e dati sensibili tramite meccanismi di sicurezza adeguati.
Controllo degli Accessi per Ruolo	L'accesso alle funzionalità del sistema deve essere regolato in base al ruolo dell'utente (amministratore o utente autenticato).
Integrità dei Dati	Il sistema deve garantire la consistenza dei dati gestiti, impedendo la creazione di issue prive dei campi obbligatori o l'inserimento di valori non validi.
Prestazioni delle Operazioni sulle Issue	Le operazioni di visualizzazione, creazione e consultazione delle issue devono essere eseguite in modo rapido ed efficiente.
Tempo di Risposta delle API	Le API REST del sistema devono fornire risposte in tempi accettabili per operazioni di autenticazione, creazione issue, commenti e aggiornamento dello stato dei bug.
Scalabilità del Sistema	L'architettura deve consentire l'aumento del numero di utenti, issue e commenti senza compromettere le prestazioni complessive del sistema.
Usabilità dell'Interfaccia	L'interfaccia utente deve essere intuitiva e facilmente utilizzabile anche da utenti con competenze informatiche limitate.
Manutenibilità del Sistema	Il sistema deve essere progettato in modo modulare e comprensibile, facilitando la manutenzione del codice e l'introduzione di nuove funzionalità.
Portabilità dell'Applicazione	L'applicazione deve poter essere eseguita su diversi ambienti software senza modifiche sostanziali, grazie all'utilizzo di tecnologie standard e containerizzazione.

4.4 Requisiti di Dominio

I requisiti di dominio descrivono i vincoli specifici del contesto applicativo di BugBoard26, derivanti dalla natura del sistema di gestione collaborativa delle issue e dalle regole operative imposte dalla traccia di progetto.

DENOMINAZIONE	DESCRIZIONE
Tipologia delle Issue	Ogni issue del sistema deve appartenere a una delle seguenti categorie previste dal dominio applicativo: *bug*, *feature*, *documentation* oppure *question*.
Campi Obbligatori della Issue	Ogni issue deve essere caratterizzata almeno da un titolo e da una descrizione, che costituiscono le informazioni minime necessarie per la sua registrazione nel sistema.
Stato Iniziale della Issue	Ogni nuova issue creata all'interno della piattaforma deve essere inizialmente impostata nello stato `TODO`.
Assegnazione dei Bug	Un membro autorizzato del progetto o un ADMIN può assegnare l'issue a un account appartenente al progetto, oppure rimuovere l'assegnazione.
Vincolo di Modifica dello Stato	La modifica è consentita ai membri del progetto e agli ADMIN. L'assegnatario autorizzato può cambiare stato, ma non dispone di un privilegio esclusivo di modifica.
Presenza della Sezione Commenti	Ogni issue deve prevedere una sezione dedicata ai commenti, nella quale gli utenti possano inserire messaggi, aggiornamenti o richieste di chiarimento.
Gestione Centralizzata dei Dati	Tutte le informazioni persistenti del sistema devono essere gestite e mantenute centralmente dal backend, che rappresenta l'unica fonte di verità per lo stato applicativo.
Indipendenza tra Frontend e Backend	Il frontend e il backend devono costituire due componenti indipendenti del sistema distribuito e devono comunicare esclusivamente attraverso interfacce esposte via rete.
Gestione dei Progetti	Il sistema deve gestire entità di tipo progetto, ciascuna delle quali rappresenta un contesto di lavoro contenente un insieme di issue correlate.
Relazione Progetto-Issue	Ogni issue appartiene a un solo progetto, mentre un progetto può contenere più issue.

4.5 Rappresentazione Tabellare Modello di A.Cockburn

Use Case 1 – Creare Issue

Use Case #N	UC1 – Creare Issue	
Goal in Context	Consentire ad un utente autenticato di creare una nuova issue nel sistema BugBoard26 per segnalare un bug, una richiesta di funzionalità o una domanda relativa al progetto software.	
Preconditions	L'utente è autenticato nel sistema. Il sistema è operativo e accessibile.	
Success End Conditions	La nuova issue è salvata nel sistema con stato iniziale `TODO` ed è visibile nell'elenco delle issue.	
Failed End Conditions	La issue non viene creata; il sistema informa l'utente dell'errore e non modifica lo stato persistente del sistema.	
Primary Actor	Utente	
Trigger	Selezione dell'azione *"Crea nuova issue"*.	
Description	Step	User Action / System
	1	L'utente seleziona l'opzione *"Crea nuova issue"*. Il sistema mostra il modulo di creazione issue.
	2	L'utente inserisce il titolo e la descrizione della issue.
	3	L'utente seleziona il tipo di issue (bug, question, documentation, feature).
	4	L'utente può specificare priorità e assegnatario facoltativi; non è presente upload immagini.
	5	L'utente conferma la creazione della issue. Il sistema valida i dati inseriti.
	6	Il sistema salva la nuova issue nel database con stato iniziale `TODO`.
	7	Il sistema mostra all'utente la pagina di dettaglio della issue appena creata.
Extensions	Step	User Action / System
	5a	Titolo o descrizione mancanti → il sistema mostra un messaggio di errore e richiede la compilazione dei campi obbligatori.
	6a	Errore durante il salvataggio → il sistema annulla l'operazione e mostra un messaggio di errore.
Subvariations	Step	User Action / System
	SV1	Assegnatario omesso → l'issue rimane non assegnata.
	SV2	L'utente non specifica priorità → il sistema conserva `null`.

Continua nella pagina successiva

Use Case #N	UC1 – Creare Issue (continua)
Notes	Immagini e storage non fanno parte del flusso attuale. Reporter, stato iniziale e date sono stabiliti dal server.

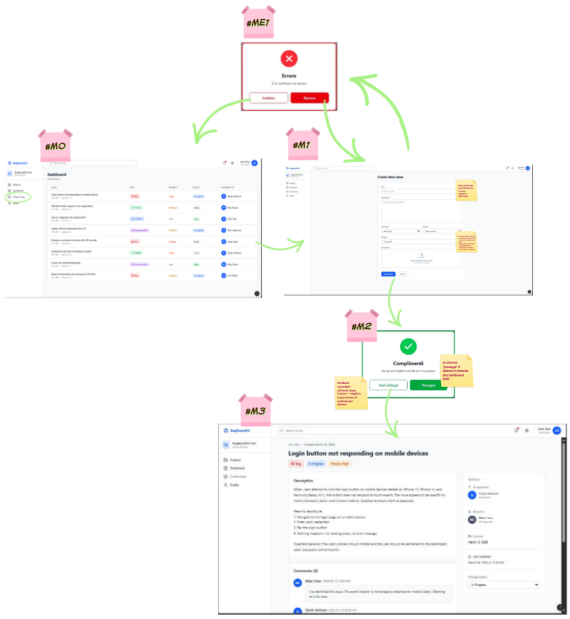


Figura 4.2: Mock-Up – Creazione Issue (disegno originale; campi e regole attuali sono descritti nel caso d'uso).

Use Case 2 – Modificare stato bug

Use Case #N	UC2 – Modificare stato bug	
Goal in Context	Consentire a un utente autorizzato del progetto o a un ADMIN di aggiornare lo stato dell'issue.	
Preconditions	Utente autenticato. Issue accessibile per membership o ruolo ADMIN; transizione ammessa.	
Success End Conditions	Lo stato del bug è aggiornato. Se risolto, viene inviata una notifica al segnalatore.	
Failed End Conditions	Lo stato non viene modificato e il sistema mostra un errore.	
Primary Actor	Membro del progetto o ADMIN	
Trigger	Selezione di "Modifica stato bug".	
Description	Step	User Action / System
	1	L'utente seleziona una issue accessibile.
	2	Il sistema mostra i dettagli del bug.
	3	L'utente seleziona la modifica dello stato.
	4	Il sistema mostra gli stati disponibili.
	5	L'utente seleziona il nuovo stato.
	6	Il sistema verifica l'autorizzazione.
	7	Il sistema aggiorna lo stato.
	8	Se risolto, il sistema invia una notifica.
	9	Il sistema conferma l'operazione.
Extensions	Step	User Action / System
	5a	Stato non valido → errore.
	6a	Utente non autorizzato → operazione negata.
Subvariations	Step	User Action / System
	SV1	Stato = `IN_PROGRESS` → notifiche di attività ai partecipanti autorizzati, escluso l'attore.
	SV2	Stato = `CLOSED` → notifica di risoluzione sempre al reporter.
Notes	Transizioni ammesse: TODO → IN_PROGRESS → CLOSED. Stesso stato idempotente; nessuna riapertura.	

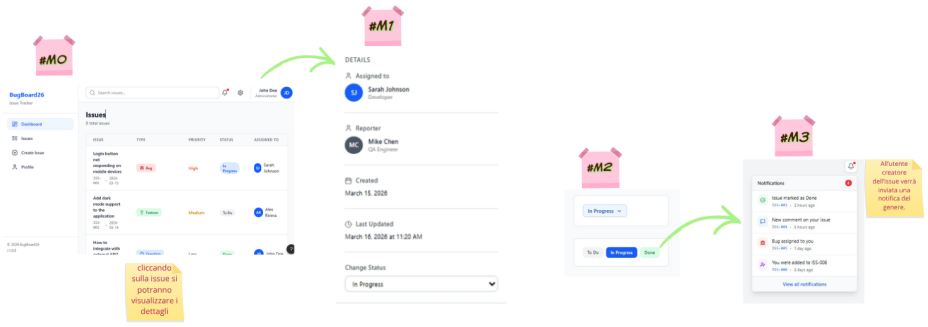


Figura 4.3: Mock-Up – Modifica Stato Bug (disegno originale; campi e regole attuali sono descritti nel caso d'uso).

Use Case 3 – Inserire un commento

Campo	**UC3 – Inserire un commento**
Goal in Context	Condividere un messaggio nel contesto di una issue.
Preconditions	Account autenticato; issue esistente e accessibile per membership o ruolo ADMIN.
Success End Conditions	Commento persistito con identità e timestamp server; elenco aggiornato.
Failed End Conditions	Nessun commento salvato; testo conservato nel campo in caso di errore API.
Primary Actor	USER membro o ADMIN.
Trigger	Invio del form nella sezione dei commenti.
Description	1. L'utente apre il dettaglio e scrive il messaggio. 2. Conferma l'invio. 3. Il backend valida accesso e contenuto. 4. Salva il commento con autore derivato dalla sessione, aggiorna l'attività e genera le notifiche previste nella stessa transazione. 5. Il frontend aggiunge il commento e svuota il campo dopo successo.
Extensions	Testo vuoto o oltre 5000 caratteri: errore di validazione. Revoca membership: accesso negato. Errore DB/notifica: rollback dell'operazione.

Campo	UC3 – Inserire un commento
Subvariations	Un errore API conserva il testo per consentire un nuovo invio.
Notes	Autore e data non sono controllabili liberamente dal browser.

Use Case 4 – Creare utenze

Use Case #N	UC4 – Creare utenze	
Goal in Context	Consentire all'amministratore di creare una nuova utenza all'interno del sistema.	
Preconditions	Utente autenticato con ruolo *ADMIN*.	
Success End Conditions	La nuova utenza viene registrata correttamente e resa disponibile nel sistema.	
Failed End Conditions	Il sistema mostra un messaggio di errore senza creare l'utenza.	
Primary Actor	Amministratore	
Trigger	Accesso alla sezione *User Management*.	
Description	Step	User Action / System
	1	L'amministratore accede alla sezione di gestione utenti.
	2	Il sistema mostra la lista degli utenti esistenti.
	3	L'amministratore seleziona l'opzione per creare un nuovo utente.
	4	Il sistema mostra il form di creazione utente.
	5	L'amministratore inserisce i dati richiesti (nome completo, email, password, ruolo).
	6	L'amministratore conferma l'inserimento.
	7	Il sistema valida i dati inseriti.
	8	Il sistema crea la nuova utenza e mostra un messaggio di conferma.
Extensions	Step	User Action / System
	7a	Dati non validi → il sistema mostra un messaggio di errore e richiede correzione.
	5a	Email già esistente → il sistema segnala il conflitto.
Subvariations	Step	User Action / System
	SV1	Creazione utente standard → ruolo USER assegnato.
	SV2	Creazione amministratore → ruolo ADMIN assegnato.
Notes	La funzione è riservata ad ADMIN. Creare un account e aggiungerlo a un progetto sono due richieste distinte: se la seconda fallisce l'account rimane creato e si può usare Add Existing User.	

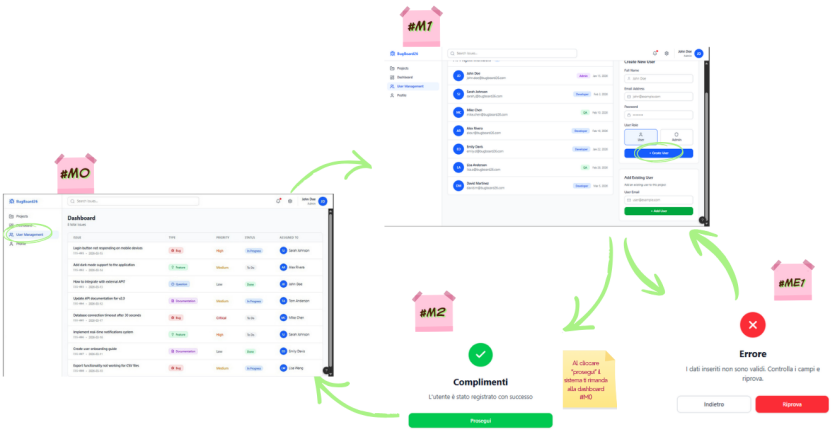


Figura 4.4: Mock-Up – Creazione Utente (disegno originale; campi e regole attuali sono descritti nel caso d'uso).

Parte II

Documento di Design del Sistema

Capitolo 5

Progettazione Architetturale e Scelte Tecnologiche

5.1 Descrizione dell'Architettura

BugBoard26 è composto da frontend React, backend REST Express e database PostgreSQL. Il browser comunica con il backend mediante HTTP/JSON; non accede direttamente al database. La separazione fra presentazione, logica applicativa e persistenza permette avvii e sviluppo indipendenti, fermo restando che i flussi reali del frontend richiedono API raggiungibili.

Frontend → Middleware/Controller → Servizi e Modelli Sequelize → PostgreSQL

5.1.1 Controller

I controller ricevono le richieste, validano i campi, leggono i modelli e costruiscono le risposte. Le operazioni composte sulle issue sono delegate ai servizi. Il controller dei commenti, invece, orchestra direttamente la transazione con i modelli e i servizi condivisi.

5.1.2 Service

I servizi condividono controlli di accesso, autenticazione, serializzazione e operazioni transazionali. `createIssue`, `changeStatus` e `changeAssignee` coordinano issue, aggiornamento progetto e notifiche. Il blocco della riga issue e l'idempotenza del cambio stato evitano duplicazioni nelle richieste concorrenti identiche.

5.1.3 Model e accesso ai dati

Non esiste un livello Repository separato. Controller e servizi usano direttamente i modelli Sequelize, che estendono `Model`. L'architettura Controller–Service–Repository del disegno iniziale va quindi letta come antecedente progettuale, non come struttura effettiva del codice. Le scritture composte e le relative notifiche condividono una transazione: un errore provoca rollback.

5.2 Architettura del Frontend

Il frontend del sistema *BugBoard26* è progettato come una **Single Page Application (SPA)** ed è responsabile della presentazione dei dati e dell'interazione con l'utente. Esso rappresenta la *View* dell'architettura MVC complessiva del sistema e comunica esclusivamente con il backend tramite API REST. La UI gestisce filtri e interazione; le regole di autorizzazione e persistenza sono applicate dal backend, garantendo una chiara separazione delle responsabilità.

Single Page Application (SPA)

Il frontend è implementato come una SPA, in cui l'applicazione viene caricata una sola volta e i contenuti vengono aggiornati dinamicamente in risposta alle azioni dell'utente. Questo approccio consente di ridurre i tempi di caricamento percepiti, evitare il ricaricamento completo delle pagine e offrire un'esperienza di navigazione più fluida e reattiva. L'aggiornamento delle viste avviene tramite chiamate asincrone al backend.

Architettura a componenti

L'interfaccia è organizzata secondo un'architettura a componenti, in cui ogni elemento dell'interfaccia utente è incapsulato in un modulo riutilizzabile. Le pagine principali dell'applicazione, come la home, la consultazione delle issue e la gestione utenti, sono composte dall'aggregazione di componenti più piccoli quali card, form e modali. Questo approccio favorisce il riuso del codice, migliora la manutenibilità e rende più semplice l'estensione futura dell'interfaccia.

Gestione della navigazione

La navigazione all'interno dell'applicazione è gestita tramite **routing client-side**, che consente di passare da una sezione all'altra senza ricaricare l'intera pagina. Ogni rotta è associata a una specifica vista e alcune sezioni sono riservate agli utenti autenticati. Il frontend gestisce quindi il flusso di navigazione e il reindirizzamento dell'utente in base allo stato di autenticazione.

Comunicazione con il backend

La comunicazione tra frontend e backend avviene esclusivamente tramite **API REST**, utilizzando richieste HTTP asincrone. Il frontend invia le richieste necessarie per recuperare o modificare i dati e riceve risposte in formato JSON, aggiornando dinamicamente l'interfaccia in base all'esito delle operazioni. Vengono inoltre gestiti gli stati di caricamento, successo ed errore, fornendo feedback visivi chiari all'utente.

Gestione dello stato

Lo stato del frontend è gestito mantenendo localmente solo le informazioni necessarie alla visualizzazione e all'interazione dell'utente, come i dati temporanei dei form o le selezioni correnti. Lo stato persistente del sistema e i dati applicativi rimangono centralizzati nel backend, che rappresenta l'unica fonte di verità del sistema. Questa scelta riduce la complessità del frontend e previene problemi di incoerenza dei dati.

Sicurezza lato Frontend

Il frontend gestisce gli aspetti di sicurezza legati alla presentazione, come la gestione dei token di autenticazione e la protezione delle sezioni riservate agli utenti autenticati. Le operazioni critiche e tutti i controlli di sicurezza reali sono comunque demandati al backend, che garantisce l'integrità e la correttezza delle operazioni.

Usabilità e responsività

Il frontend è progettato con particolare attenzione all'usabilità e alla responsività. L'interfaccia è pensata per adattarsi a diverse dimensioni di schermo e per guidare l'utente nello svolgimento delle principali operazioni, fornendo feedback visivi coerenti e chiari. Questo contribuisce a migliorare la qualità percepita del sistema e l'esperienza complessiva dell'utente.

Comportamenti attuali Le API reali sono usate di default; `VITE_USE_MOCK_API=true` abilita la simulazione, non equivalente a tutte le regole del backend. Il profilo è una schermata placeholder; la creazione issue è una modale, raggiungibile anche tramite redirect.

5.3 Criteri di Design e Motivazioni

Separazione delle responsabilità e riuso

Le pagine React compongono layout, form, commenti e pannelli riutilizzabili. Il client Axios centralizza token e gestione del 401. Nel backend i moduli di validazione, accesso, risposta e notifica sono condivisi. La separazione descrive i moduli reali senza introdurre classi Java inesistenti.

Sicurezza

JWT HS256, BCrypt tramite `bcryptjs`, validazione server e controllo della membership proteggono le operazioni. Il ruolo dell'account viene riletto a ogni richiesta; CORS limita le origini ammesse ma non sostituisce l'autorizzazione. Il logout è locale e non revoca subito il JWT.

Progettazione a oggetti e composizione

I sei modelli estendono Sequelize `Model`; `ApiError` estende `Error`. Il frontend usa componenti funzione, hook e composizione. I DTO sono contratti JSON costruiti da funzioni di serializzazione, non gerarchie di classi Java. La scelta riduce duplicazione senza attribuire al sistema un'architettura diversa da quella implementata.

Scalabilità ed esperienza utente

Separazione dei processi, componenti e stato di caricamento supportano manutenzione e interazione. Polling delle notifiche ogni 30 secondi e liste non paginate sono scelte attuali con limiti da valutare; prestazioni e usabilità non sono certificate senza misure.

5.4 Tecnologie Utilizzate

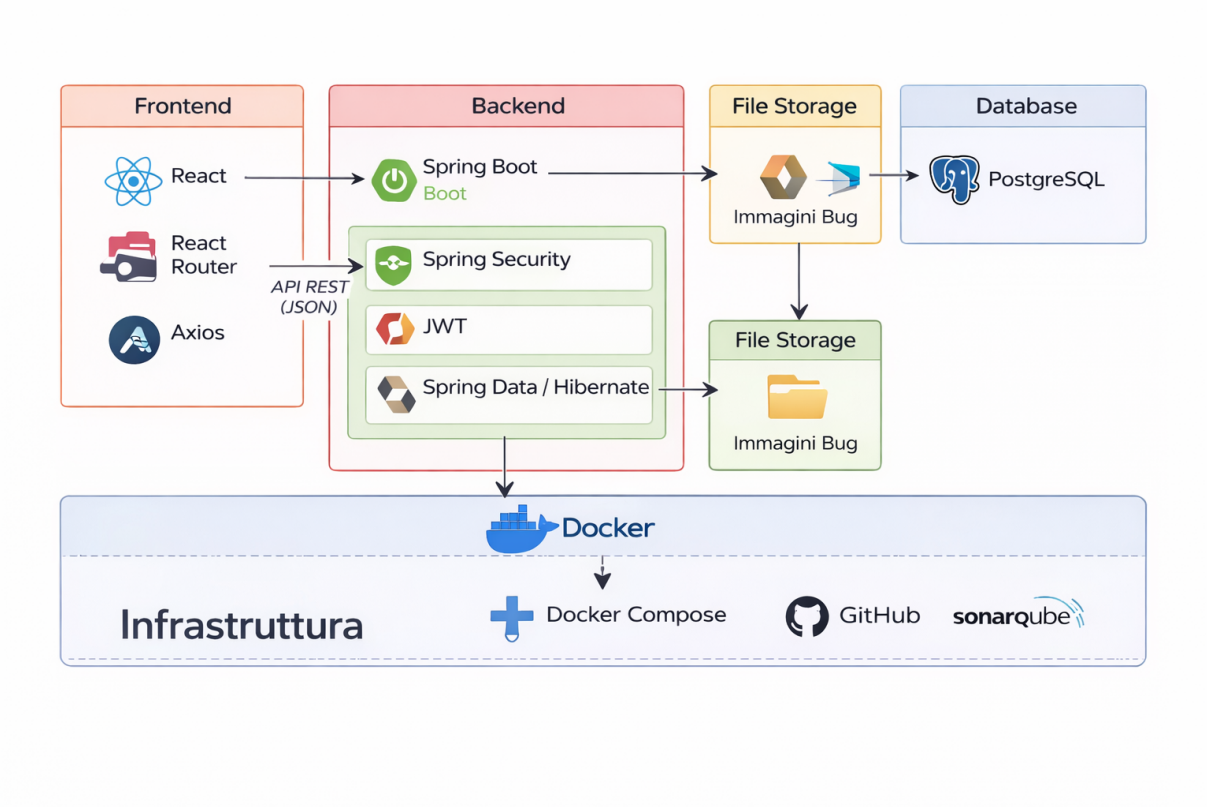


Figura 5.1: Schema tecnologico originale del progetto, conservato come elaborato grafico. I riferimenti a Spring e a servizi di storage descrivono la progettazione iniziale; lo stack corrente è quello specificato nel testo.

Frontend React 19, React Router, Axios e Vite; navigazione SPA e componenti riutilizzabili.

Backend Node.js (almeno 22), JavaScript ES modules ed Express 5. Nessun backend Spring Boot o Spring Security è presente nello snapshot corrente.

Persistenza Sequelize 6, driver `pg` e PostgreSQL; il Compose usa PostgreSQL 17.

Autenticazione JWT HS256 e hash BCrypt; il bootstrap ADMIN è disponibile in development.

File e immagini Upload e storage non sono implementati: i relativi elementi del disegno originale non vanno considerati funzionalità concluse.

Infrastruttura Dockerfile backend e Compose backend/PostgreSQL. Il frontend viene avviato con Vite; non è incluso un Dockerfile frontend.

Qualità Sono presenti 40 casi API backend, ESLint e build frontend. Report SonarQube, copertura misurata e risultati sperimentali di usabilità restano da produrre.

5.5 Schema della Persistenza dei Dati

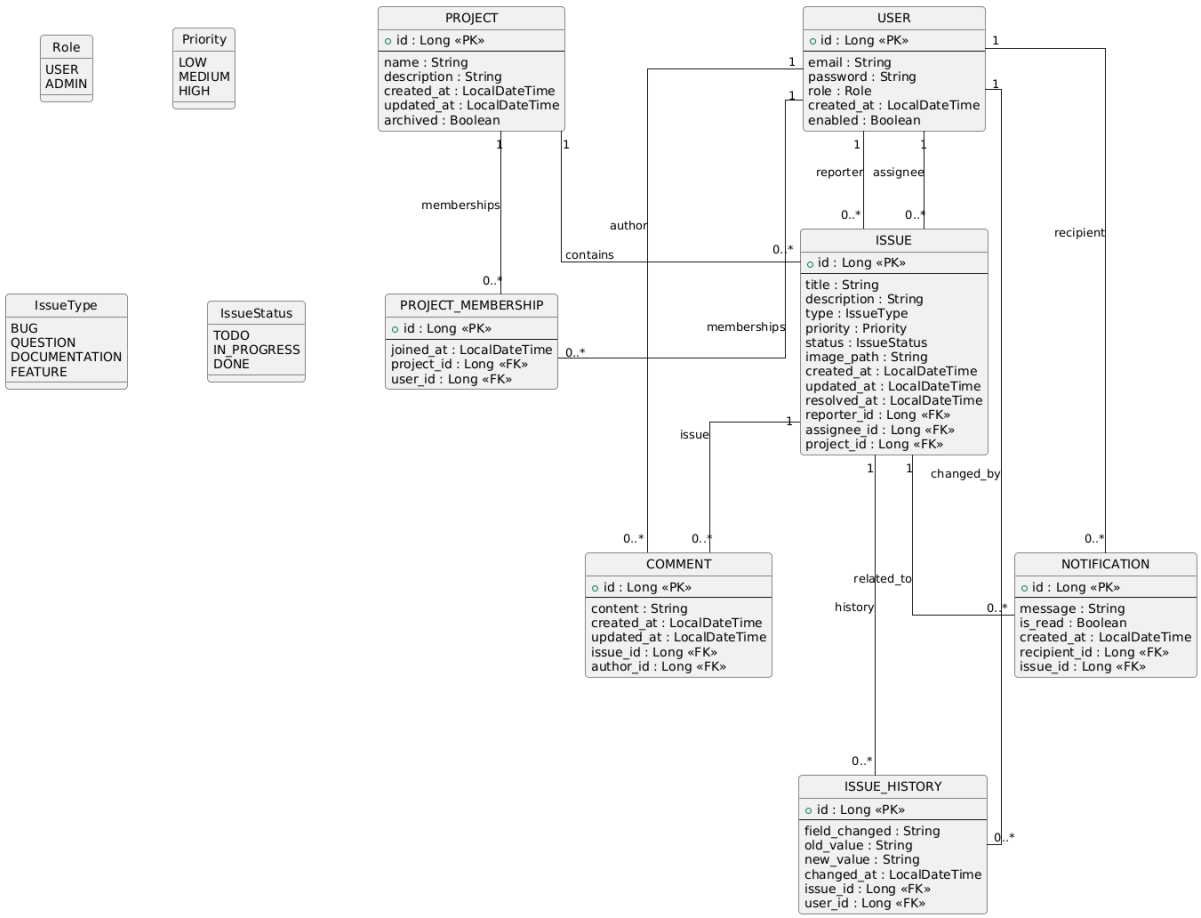


Figura 5.2: Class diagram originale del dominio, mantenuto nel formato dell'autore. Le parti progettuali non implementate, compreso ISSUE_HISTORY, sono distinte dal modello corrente nel testo.

5.5.1 Descrizione delle entità e delle relazioni

I modelli effettivi sono `User`, `Project`, `Membership`, `Issue`, `Comment` e `Notification`. User e Project sono collegati molti-a-molti tramite Membership, con coppia progetto/utente univoca. Ogni issue appartiene a un progetto e a un reporter; può avere un assegnatario, che deve essere membro del progetto. Commenti e notifiche appartengono a una issue e rispettivamente a un autore o un destinatario.

La priorità è nullable, lo stato iniziale è TODO e le transizioni sono TODO → IN_PROGRESS → CLOSED. Le notifiche persistono il tipo ACTIVITY o RESOLUTION. Una risoluzione resta leggibile al reporter ex membro: nella risposta API `issueId` può essere mascherato con `null`, senza modificare il legame nel DB. Non sono implementati ISSUE_HISTORY, allegati o generazione retroattiva delle notifiche di risoluzione.

Migrazione e compatibilità `db:init` applica 001 e 002 in transazione; il server non migra automaticamente lo schema. Lo script 002 converte OPEN in TODO, rende la priorità nullable e aggiunge il tipo notifica. Lo stato del DB di sviluppo va verificato. Il campo JSON `openIssues` mantiene il nome per compatibilità ma conta le issue TODO.

Capitolo 6

Design dell'interfaccia utente

6.1 Progettazione Cromatica dell'Interfaccia

La progettazione cromatica dell'interfaccia utente di BugBoard26 è stata affrontata come elemento fondamentale del design dell'esperienza utente e dei requisiti non funzionali del sistema.

Le scelte cromatiche adottate non hanno una finalità puramente estetica, ma contribuiscono direttamente a migliorare:

- l'usabilità dell'applicazione;
- la leggibilità dei contenuti;
- la rapidità di riconoscimento delle informazioni;
- la coerenza visiva complessiva del sistema.

In un contesto di gestione delle issue, in cui l'utente deve analizzare rapidamente informazioni e stati, il colore assume un ruolo chiave nel supportare i processi cognitivi e decisionali.

6.1.1 Obiettivi della Progettazione Cromatica

L'obiettivo principale della progettazione cromatica è quello di facilitare l'interazione dell'utente con il sistema, riducendo il carico cognitivo e migliorando l'efficienza operativa.

In particolare, la palette cromatica è stata definita per:

- trasmettere un senso di affidabilità e professionalità, in linea con strumenti di sviluppo software;
- migliorare la leggibilità dei contenuti testuali e delle informazioni strutturate;
- guidare l'attenzione verso le azioni principali (call-to-action);
- supportare il riconoscimento immediato del tipo e dello stato delle issue;
- mantenere coerenza visiva tra le diverse sezioni dell'applicazione.

6.1.2 Palette Cromatica Adottata

La palette cromatica dell'applicazione è basata su tonalità predominanti di blu, affiancate da colori neutri e colori funzionali.

Il blu è stato scelto come colore primario in quanto associato a concetti di:

- affidabilità;
- stabilità;
- competenza.

I colori neutri, come bianco e grigio chiaro, sono utilizzati per:

- sfondi;
- superfici principali;
- contenitori (card);

al fine di garantire elevata leggibilità e ridurre il rumore visivo.

Accanto alla palette principale, sono stati introdotti colori funzionali per rappresentare il tipo di issue:

- rosso → bug (errore o malfunzionamento);
- blu → question (richiesta di informazione);
- verde → feature (nuova funzionalità);
- viola → documentation (contenuti documentali).

Questa scelta consente un riconoscimento immediato del tipo di issue, migliorando l'efficienza nella consultazione delle informazioni.

6.1.3 Applicazione della Regola 60–30–10

La distribuzione cromatica dell'interfaccia di *BugBoard26* segue il principio della regola 60–30–10, adottata per garantire equilibrio visivo, gerarchia percettiva e chiarezza nella presentazione delle informazioni.

Secondo tale principio:

- il 60% dell'interfaccia è occupato da colori neutri, come bianco e grigio chiaro, utilizzati per lo sfondo generale, la topbar, la sidebar e le aree principali del contenuto;
- il 30% è riservato al colore secondario dominante, rappresentato principalmente dal blu, impiegato nel logo dell'applicazione, negli elementi di navigazione, negli avatar e in alcuni componenti interattivi, al fine di rafforzare l'identità visiva del sistema;
- il restante 10% è costituito dai colori di accento, utilizzati per evidenziare informazioni semanticamente rilevanti e attirare l'attenzione dell'utente su elementi specifici, quali:
  - tipologia delle issue;

– priorità;
– stato di avanzamento;
– pulsanti e componenti interattivi;
– notifiche visive e indicatori di feedback.

L'applicazione di questa regola consente di evitare un uso eccessivo del colore, mantenendo l'interfaccia ordinata e facilmente leggibile. Inoltre, la distinzione tra colori neutri, colore identitario e colori di accento permette di guidare l'attenzione dell'utente verso le informazioni più importanti, migliorando la navigazione e l'esperienza d'uso complessiva.

Questa distribuzione permette di guidare l'attenzione dell'utente verso gli elementi più rilevanti, migliorando l'efficacia dell'interazione.

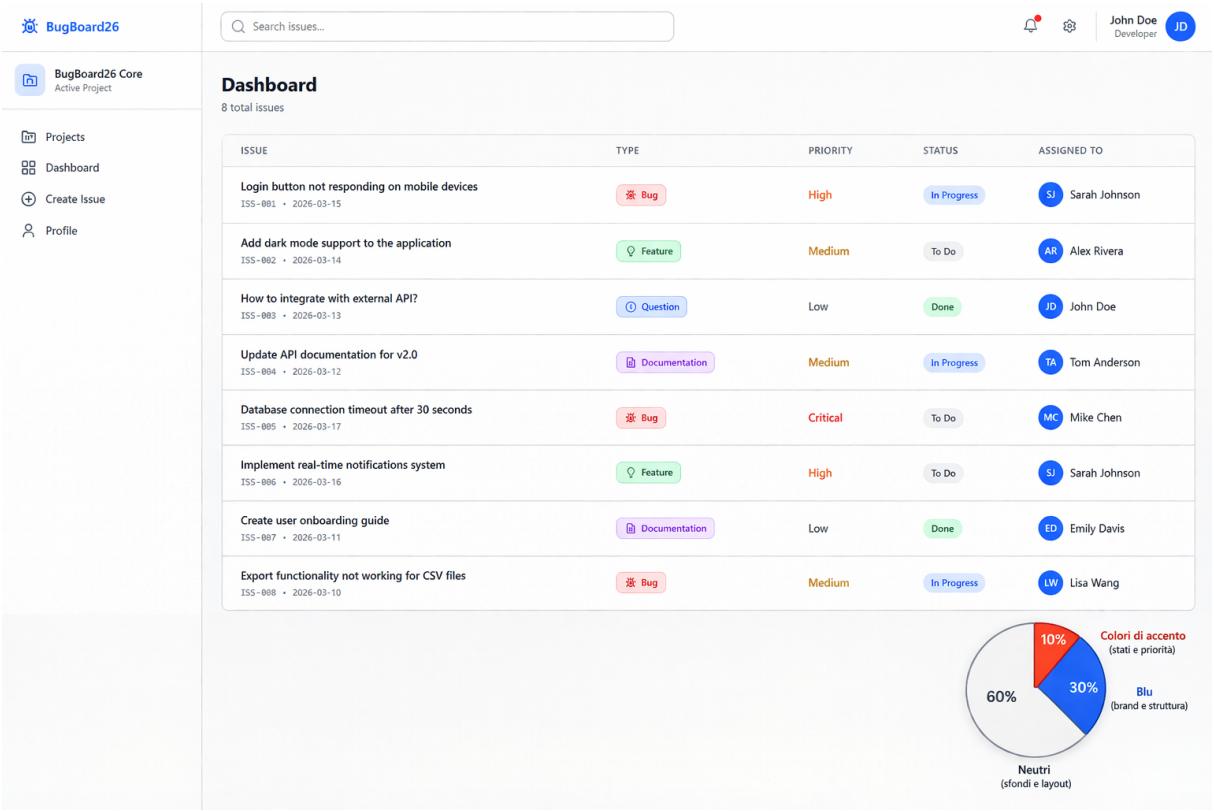


Figura 6.1: Applicazione della regola 60–30–10 nell'interfaccia di BugBoard26.

6.1.4 Accessibilità e Contrasto Cromatico

Durante la definizione della palette cromatica è stata posta particolare attenzione al rispetto delle Web Content Accessibility Guidelines (WCAG), in particolare per quanto riguarda il contrasto tra testo e sfondo.

Le combinazioni cromatiche sono state scelte con l'obiettivo di rispettare il contrasto WCAG AA; la conformità deve ancora essere verificata con misure sui colori effettivamente renderizzati. Gli obiettivi sono:

- una corretta leggibilità dei contenuti;
- accessibilità anche per utenti con difficoltà visive;

- una fruizione efficace dell'interfaccia in diversi contesti d'uso.

Inoltre, il colore non è mai utilizzato come unico mezzo per trasmettere informazioni: etichette testuali e icone supportano sempre la codifica cromatica, evitando ambiguità interpretative.

6.2 Tipografia

La progettazione tipografica dell'interfaccia utente di BugBoard26 è stata affrontata con l'obiettivo di garantire un'elevata leggibilità dei contenuti, una chiara gerarchia informativa e una coerenza visiva complessiva dell'applicazione.

In un sistema di issue tracking, la tipografia assume un ruolo fondamentale, poiché gli utenti devono poter analizzare rapidamente grandi quantità di informazioni testuali, come titoli, descrizioni e stati delle issue.

La tipografia contribuisce quindi a migliorare:

- la comprensione delle informazioni;
- la velocità di lettura;
- la riduzione del carico cognitivo durante l'interazione.

6.2.1 Obiettivi della Progettazione Tipografica

L'obiettivo principale della progettazione tipografica è quello di supportare l'utente nell'interpretazione dei contenuti, facilitando la navigazione e la consultazione delle informazioni.

In particolare, la tipografia è stata progettata per:

- migliorare la leggibilità dei testi su dispositivi desktop e mobile;
- permettere una rapida scansione visiva delle liste di issue;
- stabilire una gerarchia chiara tra titoli, sottotitoli e contenuti;
- ridurre il carico cognitivo durante la consultazione;
- mantenere coerenza grafica in tutte le schermate dell'applicazione.

6.2.2 Scelta del Font

È stato adottato un font di tipo sans-serif, particolarmente indicato per le interfacce digitali grazie alla sua struttura semplice e lineare.

I font sans-serif risultano più leggibili su schermi retroilluminati e mantengono una buona resa anche a dimensioni ridotte, caratteristica fondamentale per un'applicazione web responsive.

Questa scelta è coerente con le principali piattaforme di issue tracking, come GitHub e Jira, che utilizzano anch'esse font sans-serif per garantire chiarezza e leggibilità.

L'utilizzo di un unico font principale contribuisce inoltre a rafforzare la coerenza visiva dell'interfaccia e a trasmettere una percezione di professionalità e affidabilità.

6.2.3 Gerarchia Tipografica

La gerarchia tipografica è stata definita in modo da rendere immediatamente distinguibili i diversi livelli informativi presenti nell'interfaccia.

Attraverso variazioni di dimensione, peso e spaziatura del testo, l'utente è guidato nella lettura secondo un ordine logico e naturale.

In particolare:

- i titoli principali identificano le sezioni e le pagine dell'applicazione;
- i titoli delle issue evidenziano le informazioni più rilevanti;
- i sottotitoli organizzano i contenuti all'interno delle pagine;
- il testo principale descrive le issue e le relative informazioni;
- i testi secondari e di supporto includono etichette, stati e metadati.

Questa organizzazione favorisce una rapida scansione visiva e consente all'utente di individuare rapidamente le informazioni di interesse.

6.2.4 Spaziatura e Allineamento

Particolare attenzione è stata dedicata alla spaziatura tra righe e blocchi di testo, al fine di evitare un'eccessiva densità visiva e migliorare la leggibilità.

Un'adeguata spaziatura verticale consente di separare chiaramente le diverse sezioni e migliora la struttura complessiva dell'interfaccia.

L'allineamento dei testi segue criteri di coerenza e regolarità, contribuendo a rafforzare la percezione di ordine e facilitando la navigazione.

6.2.5 Accessibilità e Conformità alle WCAG

La progettazione tipografica è stata realizzata tenendo conto delle Web Content Accessibility Guidelines (WCAG), in particolare per quanto riguarda:

- dimensione minima del testo;
- contrasto tra testo e sfondo;
- leggibilità su diversi dispositivi.

Queste scelte perseguono una fruizione adeguata; contrasto, tastiera e tecnologie assistive richiedono una verifica specifica prima di dichiarare conformità.

6.2.6 Coerenza con l'Identità Visiva

La tipografia adottata si integra in modo coerente con la palette cromatica definita nella sezione precedente.

L'insieme delle scelte tipografiche e cromatiche contribuisce a trasmettere:

- chiarezza;
- affidabilità;
- efficienza operativa;

valori fondamentali per una piattaforma di gestione delle issue come BugBoard26.

6.3 Principles of Proximity and Connectedness

I principi di prossimità e connessione, derivanti dalla psicologia della Gestalt, rappresentano due linee guida fondamentali nella progettazione di interfacce utente efficaci. Essi consentono di organizzare visivamente le informazioni, facilitando la comprensione dell'interfaccia e riducendo il carico cognitivo dell'utente.

Principio di Prossimità

Secondo il principio di prossimità, elementi che si trovano fisicamente vicini tra loro vengono percepiti come appartenenti allo stesso gruppo funzionale. Questo meccanismo percettivo permette di strutturare le informazioni senza l'uso eccessivo di separatori grafici o decorazioni aggiuntive.

All'interno di *BugBoard26*, il principio di prossimità viene applicato per:

- raggruppare campi di input correlati all'interno dei form;
- associare etichette, valori e pulsanti di azione;
- suddividere visivamente le informazioni all'interno delle stesse schermate.

Tale scelta consente all'utente di comprendere rapidamente la struttura dei contenuti e di individuare con facilità gli elementi tra loro correlati.

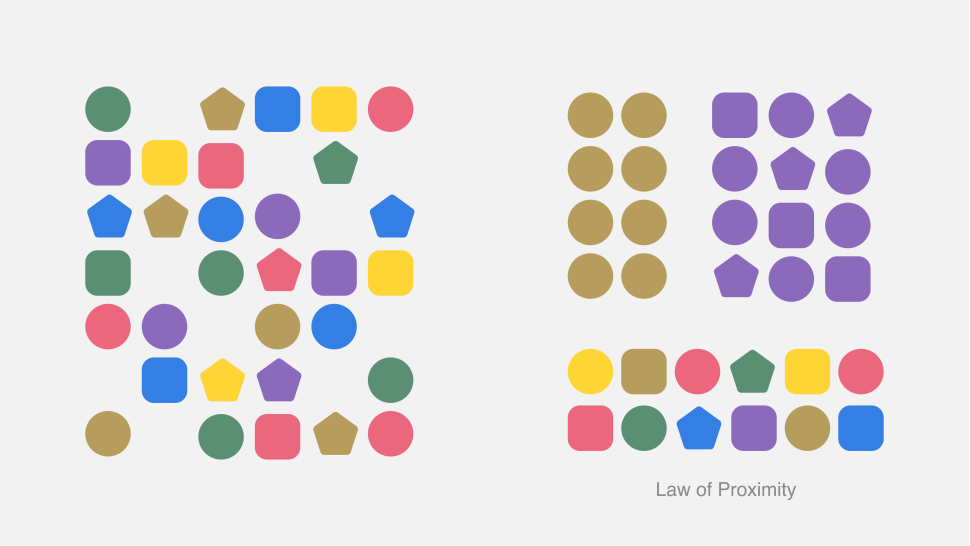


Figura 6.2: Esempio di applicazione del principio di prossimità

Principio di Connectedness

Il principio di connectedness afferma che elementi visivamente collegati tra loro tramite bordi, sfondi, linee o colori comuni vengono percepiti come appartenenti allo stesso insieme, anche in assenza di una forte prossimità spaziale.

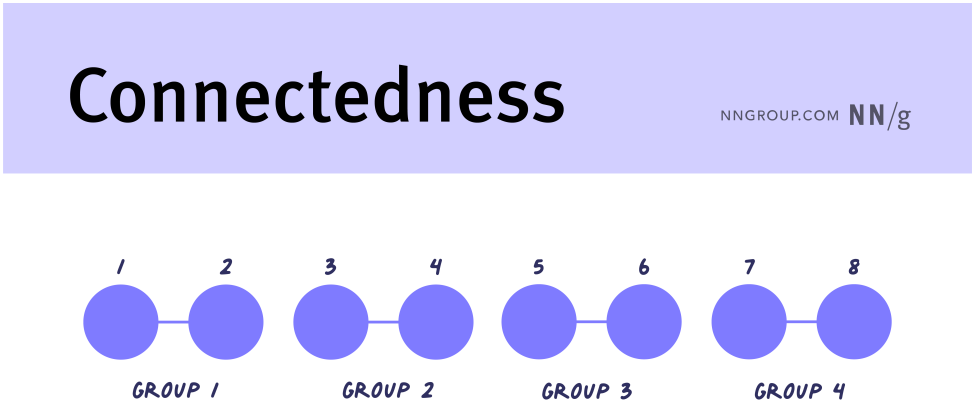


Figura 6.3: Esempio di applicazione del principio di prossimità

Nel sistema, questo principio viene applicato attraverso:

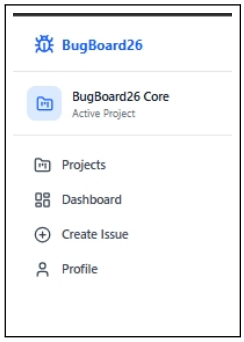
- l'utilizzo di contenitori grafici comuni, come card e pannelli;
- l'impiego coerente di colori e sfondi per sezioni omogenee;
- la delimitazione visiva delle aree funzionali principali.

La connectedness rafforza il principio di prossimità, rendendo ancora più immediata la comprensione delle relazioni tra gli elementi dell'interfaccia.

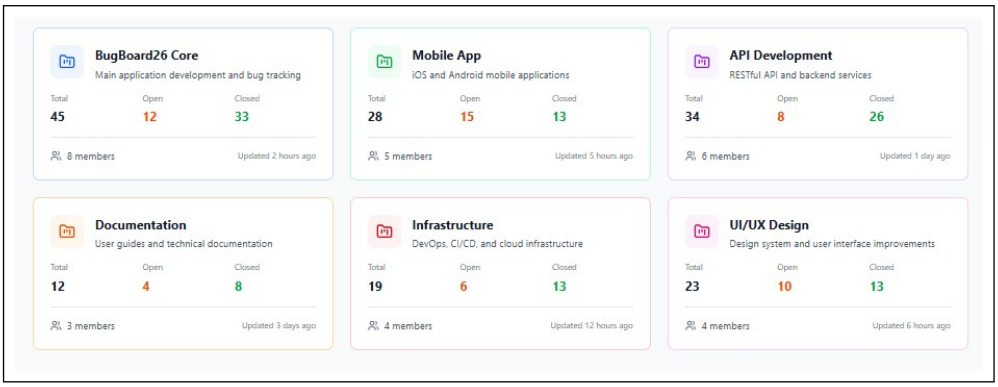
Applicazione nell'Interfaccia

L'uso combinato dei principi di prossimità e connessione è particolarmente evidente in alcune sezioni chiave dell'interfaccia di *BugBoard26*, contribuendo a migliorare la comprensione e la navigazione all'interno del sistema.

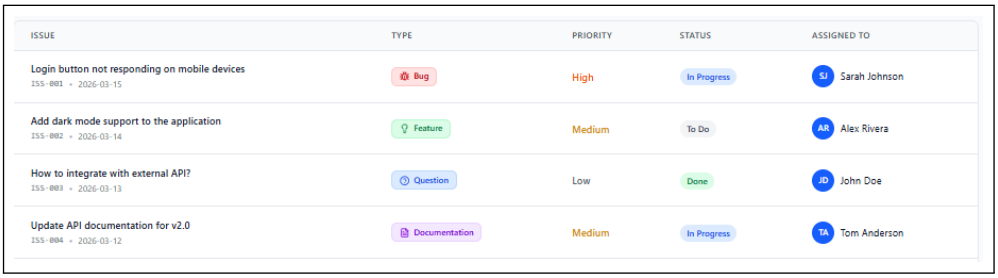
- **Barra di navigazione**: tutti gli elementi del menu, inclusa l'icona del profilo utente, sono percepiti come parte di un unico insieme grazie alla loro vicinanza, al colore condiviso e alla collocazione in una regione comune distinta dal resto della pagina, facilitando l'accesso rapido alle principali funzionalità.



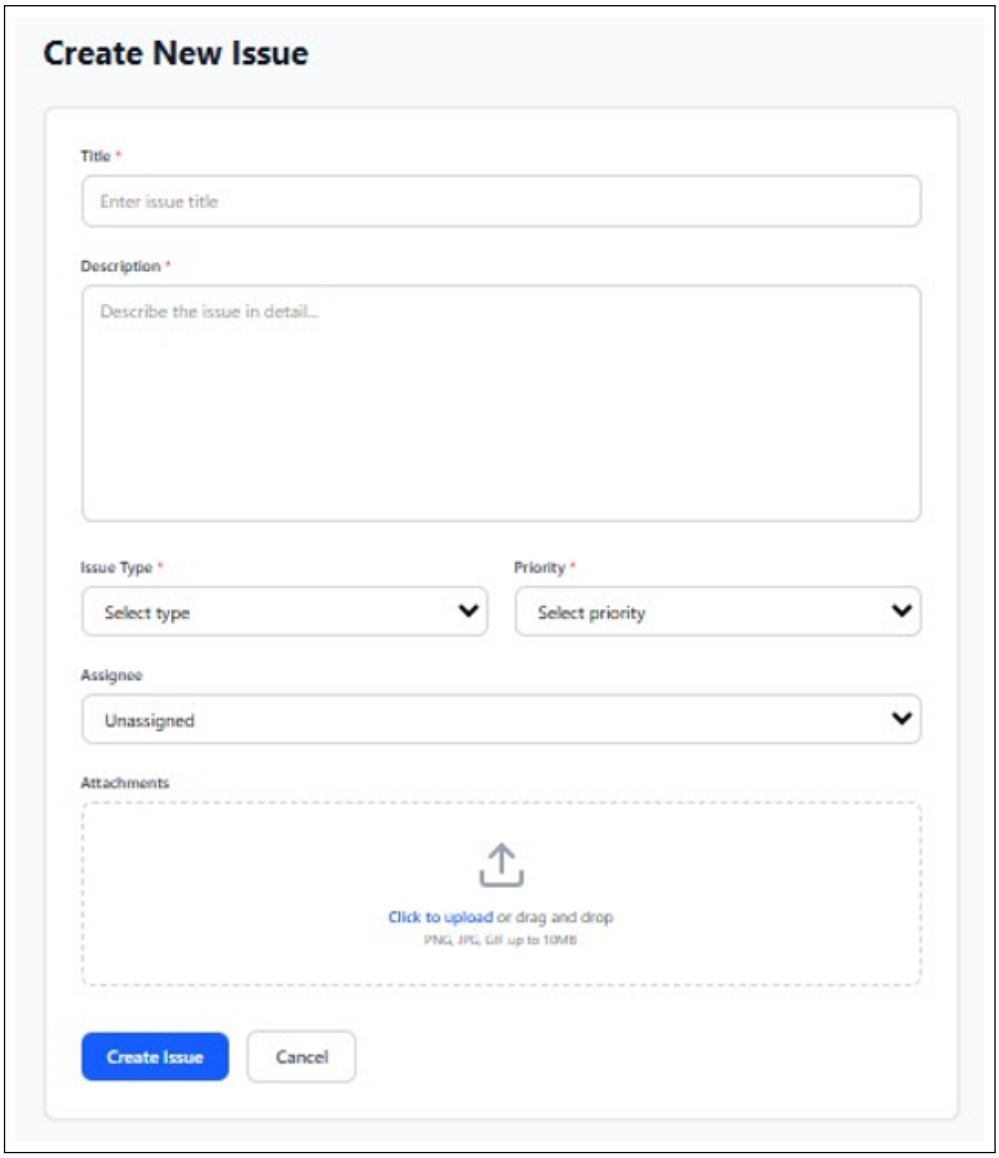
- **Dashboard dei progetti**: i progetti sono rappresentati tramite card visivamente separate ma coerenti, ciascuna contenente informazioni sintetiche (nome, descrizione, stato). La disposizione spaziale e la struttura uniforme consentono all'utente di individuare rapidamente il progetto di interesse.



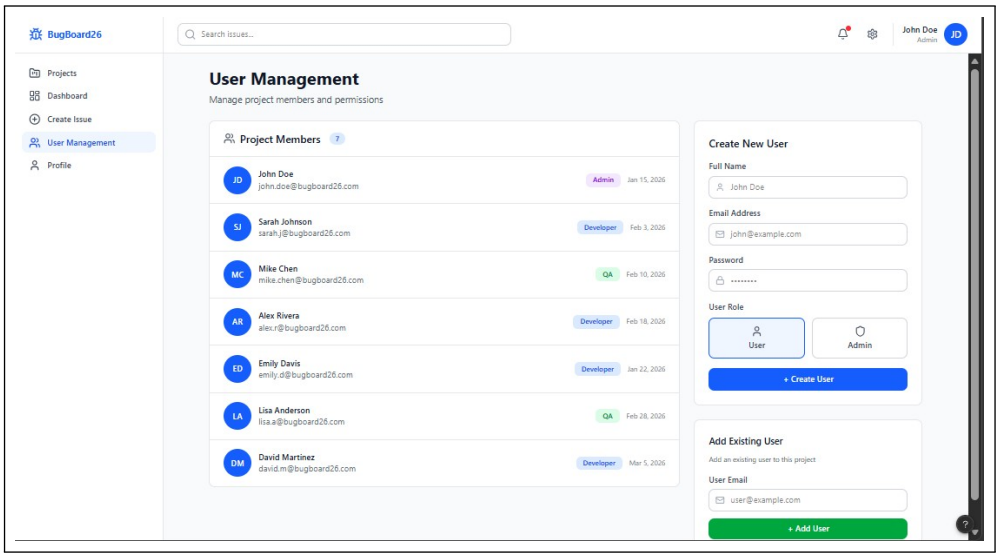
- **Vista delle issue**: le issue sono organizzate in liste o board, in cui ogni elemento è racchiuso in un contenitore visivo che raggruppa titolo, priorità, stato e assegnatario. Questo favorisce una lettura immediata e il confronto tra più segnalazioni.



- **Form di creazione delle issue**: i campi sono suddivisi in gruppi logici (informazioni principali, dettagli, assegnazione), separati visivamente ma coerenti internamente, facilitando la compilazione e riducendo il carico cognitivo.



- **Form di creazione di un nuovo utente e dell'aggiunta di un utente esistente ad un progetto**: I due campi sono posizionati al lato destro della pagina di gestione degli utenti di un progetto uno sotto l'altro in modo da essere associati a due funzionalità dello stesso genere.



Benefici sull'Usabilità

L'adozione dei principi di prossimità e connectedness contribuisce in modo significativo a migliorare l'usabilità complessiva del sistema BugBoard26, in particolare nella gestione dei progetti e delle relative issue.

In questo contesto, tali principi permettono di:

- organizzare in modo chiaro i progetti all'interno della dashboard iniziale, facilitando l'accesso rapido alle aree di lavoro;
- raggruppare visivamente le informazioni rilevanti di una issue (titolo, stato, priorità, assegnatario), migliorandone la leggibilità;
- ridurre gli errori di interpretazione durante la navigazione tra progetti e issue;
- diminuire il carico cognitivo dell'utente, grazie a una struttura coerente e prevedibile dell'interfaccia;
- rendere più immediata la comprensione delle relazioni tra elementi (issue, commenti, notifiche).

Queste scelte progettuali favoriscono una percezione di ordine e coerenza, consentendo agli utenti di orientarsi rapidamente all'interno del sistema e di interagire in modo efficiente con le funzionalità offerte, con l'obiettivo di migliorare complessivamente l'esperienza d'uso; l'effetto deve essere verificato con utenti.

6.4 Applicazione del modello F-Pattern

Nella progettazione dell'interfaccia utente del sistema BugBoard26 è stato adottato il modello di lettura **F-pattern**, che descrive il comportamento tipico degli utenti durante la scansione visiva di una pagina web.

Come mostrato in Figura 6.4, il percorso visivo dell'utente segue una forma a "F", articolata nei seguenti passaggi:

- $(1 \rightarrow 2)$ scansione del titolo e del contesto nella parte superiore della dashboard.
- $(1 \rightarrow 3 \rightarrow 5)$ lettura verticale dei punti di ingresso e delle informazioni principali.
- $(3 \rightarrow 4)$ consultazione orizzontale dei dettagli e dei controlli associati al contenuto selezionato.

La disposizione mira a facilitare la consultazione. La figura conserva la proposta originale; non sono stati svolti eye-tracking o misure sperimentali sul percorso dello sguardo.

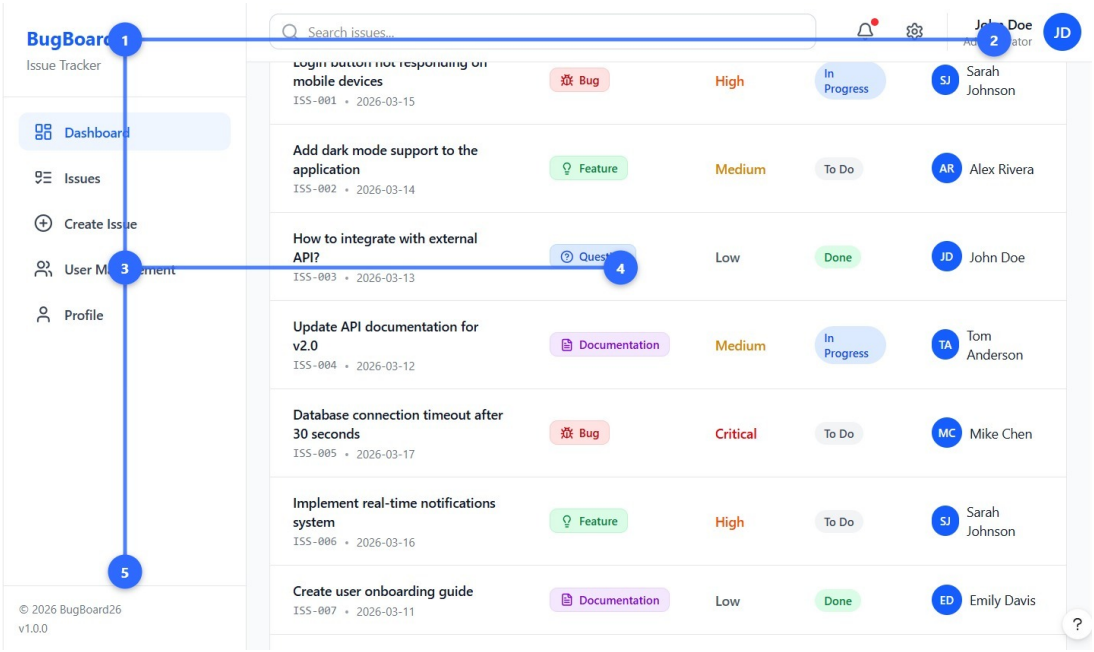


Figura 6.4: Applicazione del modello F-pattern nella dashboard di BugBoard26

6.5 Labor Perception Bias

Messaggi di caricamento, pulsanti *Creating...* e controlli disabilitati durante la richiesta rendono percepibile il lavoro del sistema. Non viene simulato un avanzamento né introdotta un'attesa artificiale; l'effetto sulla fiducia dell'utente è da valutare sperimentalmente.

6.6 Gestione del Feedback Utente e Feedback Persistence

Sono conservati i mockup originali di conferma ed errore come proposta visiva. Nel codice attuale il successo di creazione chiude la modale e apre il dettaglio, senza una modale separata di congratulazioni. L'errore resta nel form; per i commenti falliti il testo viene conservato. La gestione utenti mostra messaggi di successo o errore fino alla nuova azione.



Figura 6.5: Mockup originali di feedback: proposte grafiche, non attestazioni del comportamento attuale.

6.7 Prevenire gli Errori

È scontato affermare che evitare un errore è più semplice che gestirlo una volta commesso; per questo motivo, ove possibile, l'interfaccia è stata progettata in modo da non mettere l'utente nelle condizioni di eseguire azioni critiche in maniera accidentale.

Nel caso di BugBoard26, questo principio è stato applicato in particolare alle operazioni distruttive o comunque sensibili, che possono modificare in modo rilevante lo stato del sistema o l'organizzazione del lavoro all'interno di un progetto.

Il mockup seguente illustra la rimozione di un progetto nella proposta iniziale. L'operazione non è implementata nello snapshot; la figura è mantenuta come progetto di interazione per un eventuale sviluppo futuro. In questo modo si riduce il rischio di cancellazioni involontarie dovute a clic errati o distrazioni, inducendo l'utente a riflettere prima di confermare l'azione.

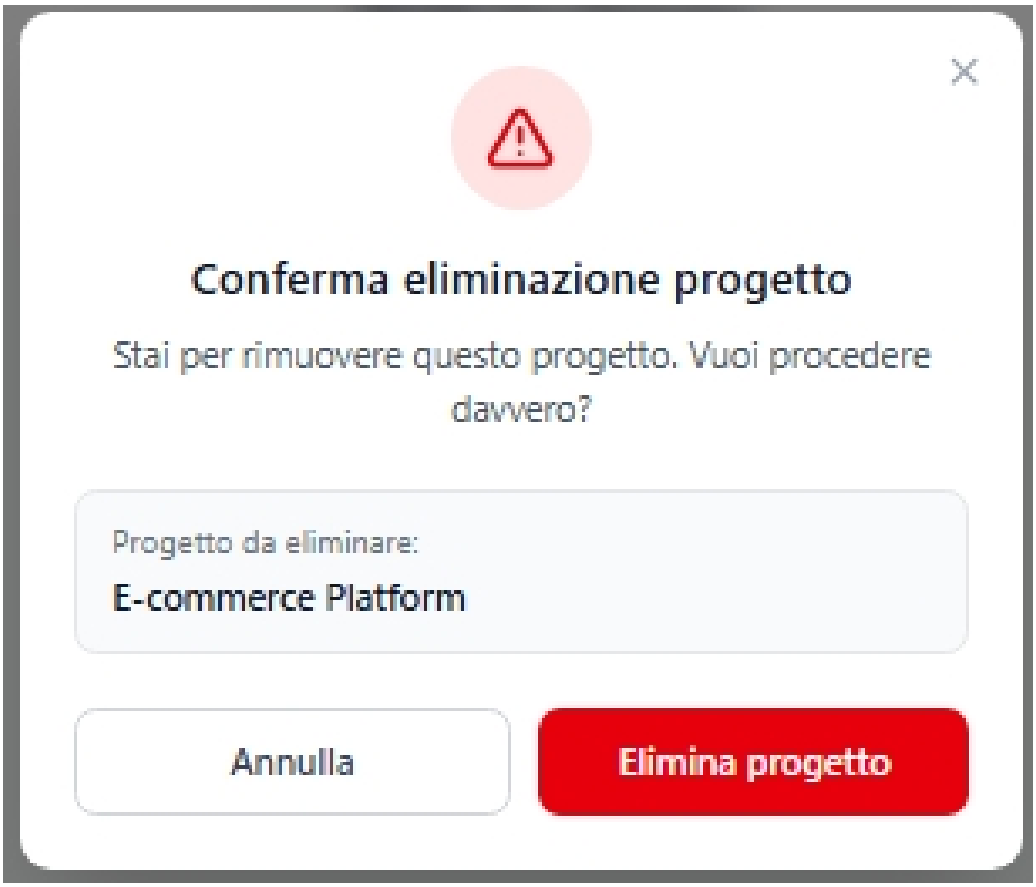


Figura 6.6: Mockup originale: proposta di conferma per la rimozione progetto, funzione non implementata.

La rimozione di un membro è invece implementata e richiede conferma tramite `window.confirm`. Il disegno seguente conserva la proposta grafica originale, diversa dalla conferma nativa usata dal codice. La rimozione di un membro, infatti, non rappresenta una normale interazione di navigazione, ma una modifica significativa nella gestione collaborativa del progetto.

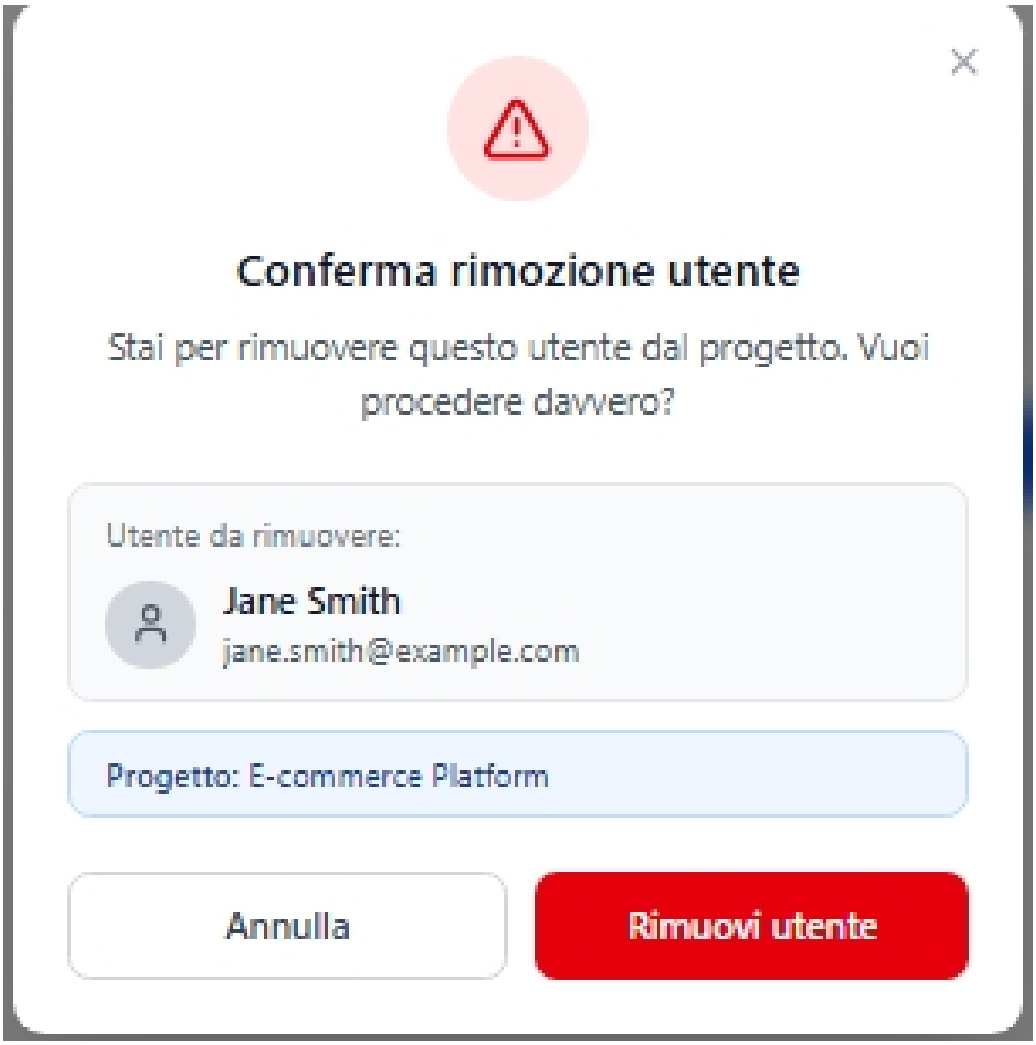


Figura 6.7: Mockup originale di conferma per la rimozione membro; il codice usa una conferma nativa.

L'adozione di queste finestre di conferma consente quindi di prevenire errori prima che essi si verifichino, invece di limitarsi a segnalarli successivamente. Tale scelta migliora il controllo percepito da parte dell'utente e contribuisce a rendere l'interazione più sicura, affidabile e coerente con la natura gestionale dell'applicazione.

Capitolo 7

Diagrammi delle classi di Design

7.1 Classi di Design del Backend

7.1.1 Architettura Controller–Service–Model

I controller di auth, users, projects, members, issues, comments e notifications espongono le API. I moduli `services/access`, `services/issues`, `services/notifications`, `services/auth` e `services/responses` condividono regole e operazioni. Il modello è costituito dalle sei classi Sequelize. Il class diagram del dominio originale è conservato nella sezione sulla persistenza. Non vengono inseriti nuovi disegni sostitutivi: per allineare i diagrammi grafici al codice, resta necessario aggiornare le parti storiche nel loro file sorgente originale.

Modulo	Operazioni e responsabilità
issues.controller	list, get, create, status, assign; validazione e risposta HTTP.
services/issues	createIssue, changeStatus, changeAssignee e touchProject; transazioni condivise.
services/access	requireProject, requireIssue, requireAssignee e visibleProjectIds.
services/notifications	notify, notifyAssignment, notifyParticipants e notifyResolution.
models/index	User, Project, Membership, Issue, Comment e Notification; mapping ORM.

7.1.2 DTO

Il diagramma DTO rappresenta gli oggetti di trasferimento dati utilizzati per lo scambio di informazioni tra frontend e backend all'interno di *BugBoard26*. Tali oggetti consentono di separare la struttura dei dati trasmessi dall'implementazione interna delle entità applicative, rendendo più chiara e controllata la comunicazione tra i diversi layer del sistema.

Il diagramma originale raggruppa i contratti di scambio in tre categorie. Nel backend Express questi oggetti sono JSON e funzioni di serializzazione, non classi Java:

- **Request DTO**, utilizzati per modellare i dati inviati dal client nelle operazioni di autenticazione, creazione e aggiornamento;
- **Domain DTO**, che rappresentano le principali informazioni di dominio gestite dall'applicazione, come utenti, progetti, issue, commenti e notifiche;
- **Response DTO**, impiegati per strutturare le risposte restituite dal backend al frontend.

Questa suddivisione permette di mantenere il sistema modulare, migliorare la leggibilità dell'architettura e favorire una gestione più ordinata dello scambio dati tra interfaccia utente e servizi applicativi.

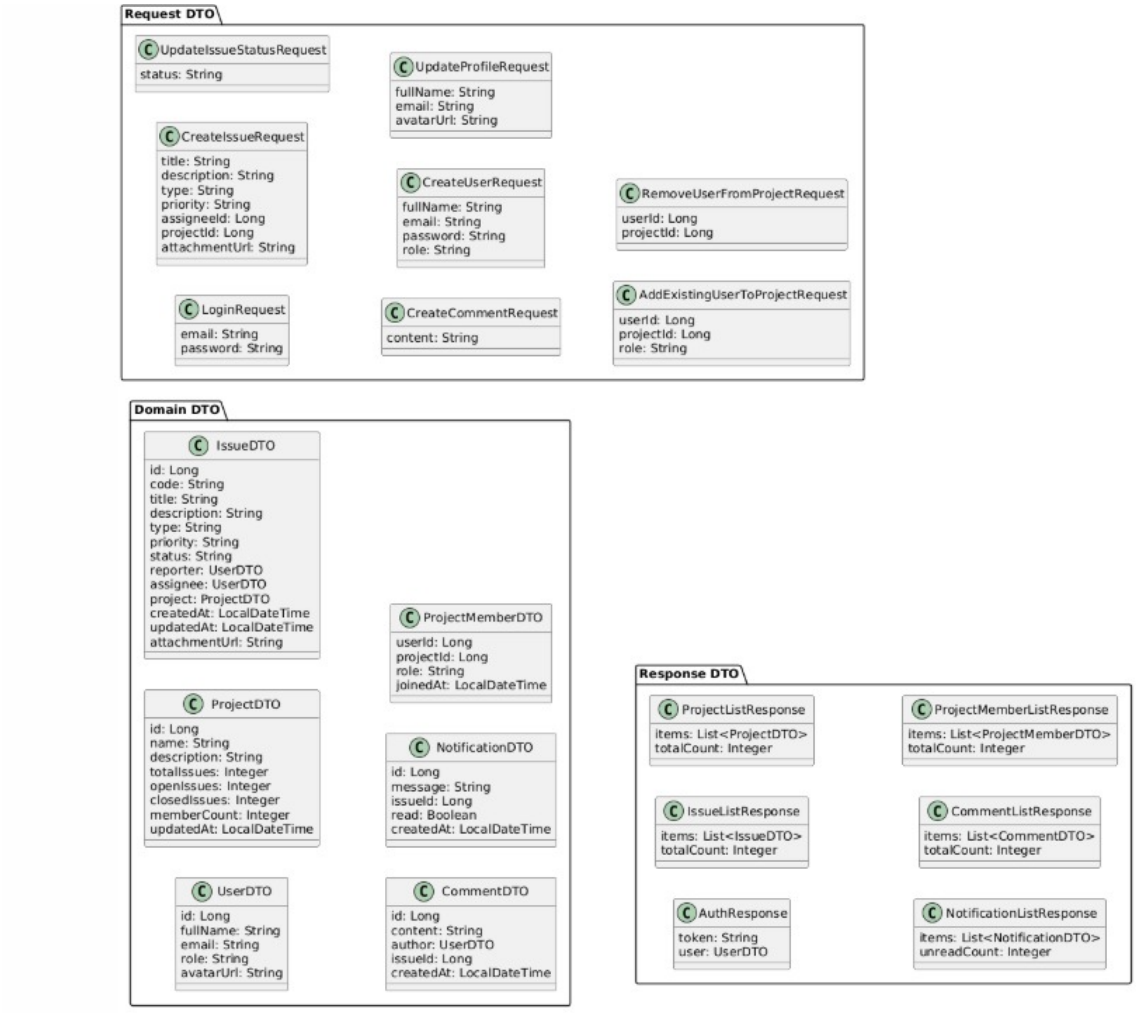


Figura 7.1: Diagramma DTO originale. Nomi e strutture di progetto sono conservati; il contratto API corrente è definito dai controller e da services/responses.js.

7.1.3 Entità di Dominio e sicurezza

Le classi persistenti estendono `Model`. Gli altri moduli backend sono funzioni ES modules; `ApiError` estende `Error`. Password e JWT sono trattati in services/auth, mentre il

middleware rilegge l'account e verifica i privilegi. I riferimenti a repository Java o JPA nei disegni storici non rappresentano l'implementazione attuale.

7.2 Classi di Design del Frontend

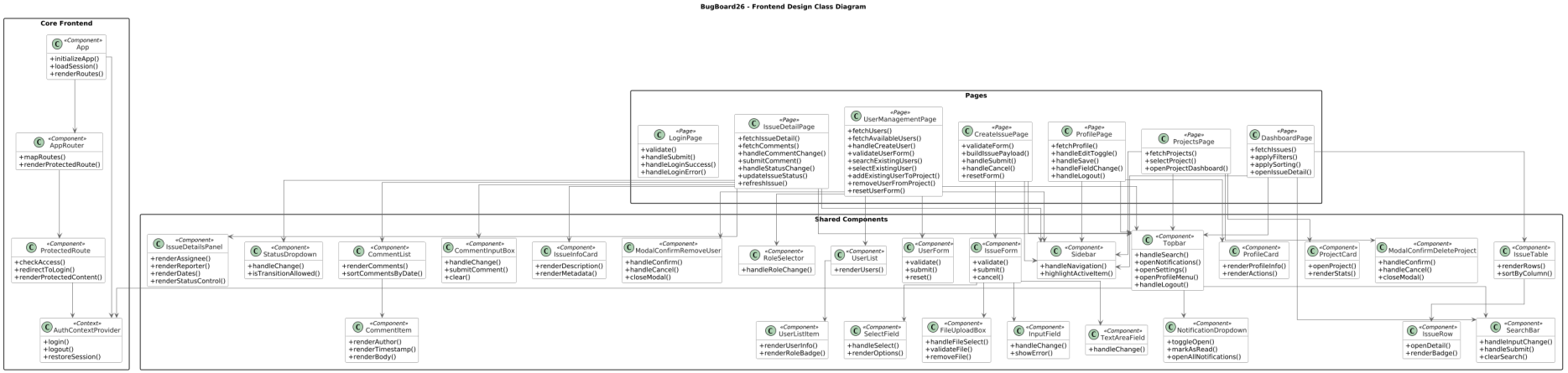


Figura 7.2: Class diagram originale del frontend: componenti e dipendenze del progetto.

Il diagramma delle classi di design del frontend rappresenta la struttura della Single Page Application di BugBoard26, evidenziando la suddivisione tra componenti core, pagine e componenti condivisi. Nel blocco Core Frontend sono stati modellati i componenti responsabili dell'inizializzazione dell'applicazione, della gestione del routing e del controllo degli accessi alle aree protette. Nel blocco Pages sono invece presenti le principali schermate del sistema, come login, selezione dei progetti, dashboard, dettaglio issue e gestione utenti. La creazione è una modale; il profilo è un placeholder, pur figurando nella vista progettuale originale. Infine, il blocco Shared Components raccoglie gli elementi riutilizzabili dell'interfaccia, come sidebar, topbar, tabella delle issue, form, modali e componenti per la gestione dei commenti.

Il diagramma è stato costruito mostrando principalmente i metodi esposti dai componenti, in modo da mantenere una rappresentazione chiara e coerente con la natura del frontend React, dove l'aspetto più rilevante è il comportamento dei componenti e le responsabilità funzionali associate. Le relazioni tra le classi descrivono soprattutto legami di composizione e utilizzo: le pagine principali si appoggiano ai componenti condivisi per realizzare le diverse funzionalità dell'interfaccia, mantenendo così modularità, riuso e separazione delle responsabilità.

7.2.1 Utility, Config e Security

Il diagramma *Utility, Config e Security* descrive l'insieme delle moduli di supporto che forniscono funzionalità trasversali al frontend di *BugBoard26*. Questi elementi non rappresentano direttamente pagine o componenti dell'interfaccia, ma costituiscono il livello infrastrutturale che consente al frontend di gestire routing, sessione utente, accesso alle API, configurazioni centralizzate e funzionalità riutilizzabili.

La sezione **Utility** raccoglie i moduli che offrono metodi di supporto utilizzati in più parti dell'applicazione. In particolare, tali classi permettono di centralizzare operazioni comuni come formattazione di valori, gestione di chiavi di memorizzazione locale e altre elaborazioni ausiliarie, evitando duplicazione di codice nei componenti di interfaccia.

La sezione **Config** contiene invece i moduli responsabili della configurazione dell'applicazione. In questo gruppo rientrano le strutture che definiscono le route del frontend, i parametri di accesso alle API e le costanti di sistema condivise. Questa scelta consente di mantenere separati i dettagli di configurazione dalla logica delle pagine e dei componenti, migliorando manutenibilità ed estendibilità.

La sezione **Security** comprende i componenti dedicati alla gestione della sessione e della protezione delle aree riservate. In particolare, vengono modellati gli elementi necessari per memorizzare e recuperare le informazioni di autenticazione, gestire il token utente e controllare l'accesso alle pagine protette attraverso meccanismi di *route guarding*. In questo modo, la logica di autenticazione e autorizzazione risulta centralizzata e riutilizzabile.

Infine, il diagramma mostra anche il collegamento con il livello **API**, che rappresenta il punto di accesso ai servizi remoti del backend. Tale organizzazione evidenzia come le configurazioni, la sicurezza e le utility costituiscano il supporto tecnico su cui si basa l'interazione tra frontend e backend.

Nel complesso, questa suddivisione permette di separare chiaramente la logica di presentazione dalla logica infrastrutturale, migliorando modularità, riuso del codice e chiarezza architetturale.

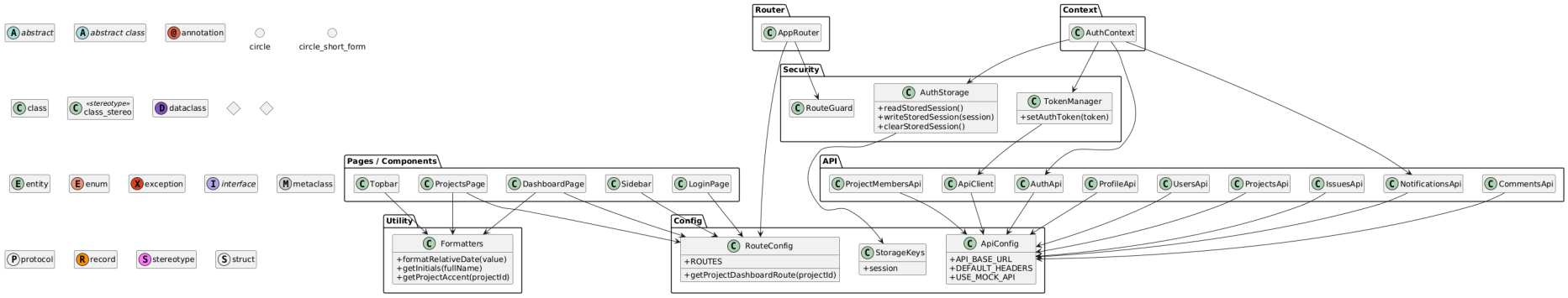


Figura 7.3: Diagramma originale Utility, Config e Security del frontend.

7.3 Gestione della navigazione tra le pagine

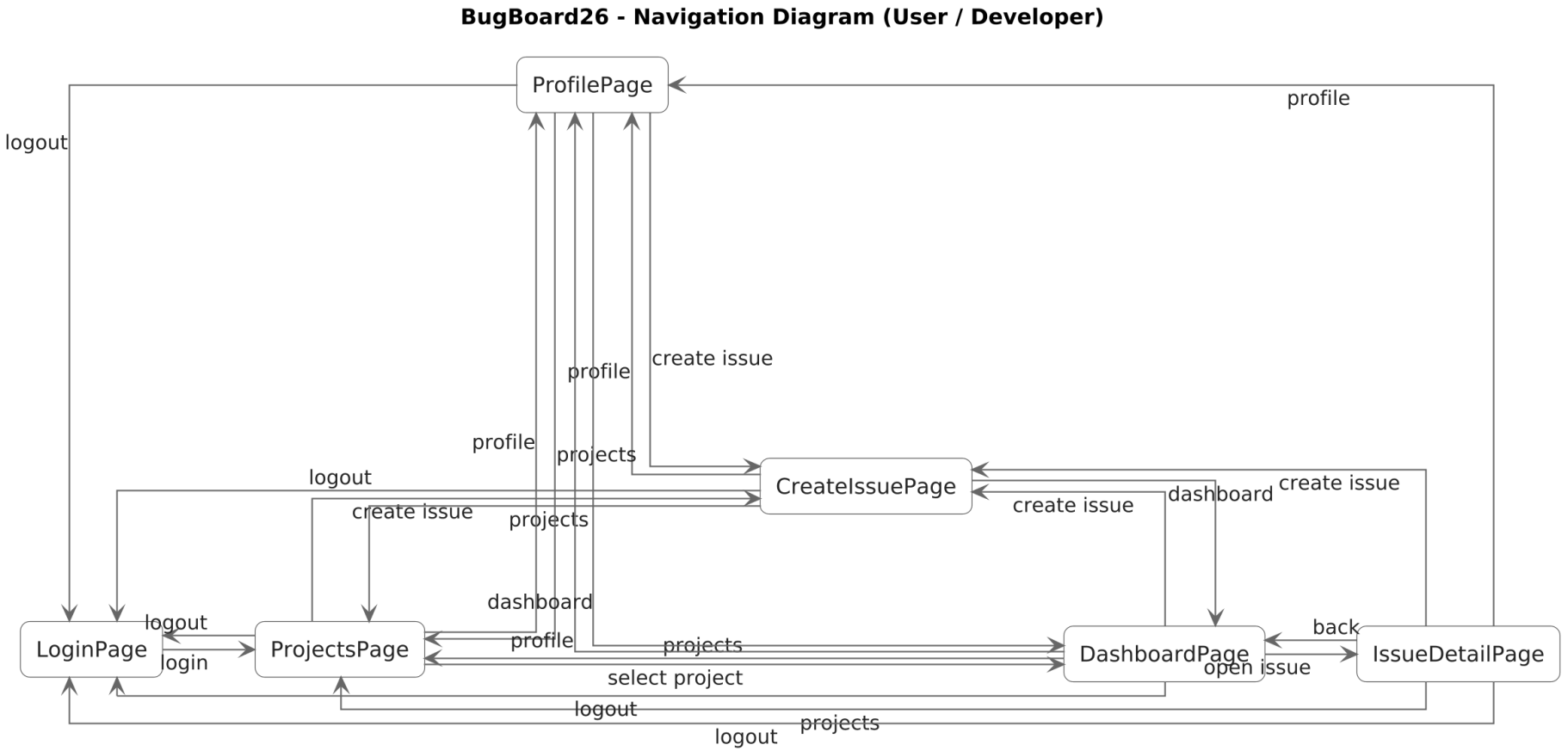


Figura 7.4: Schema originale della navigazione – User.

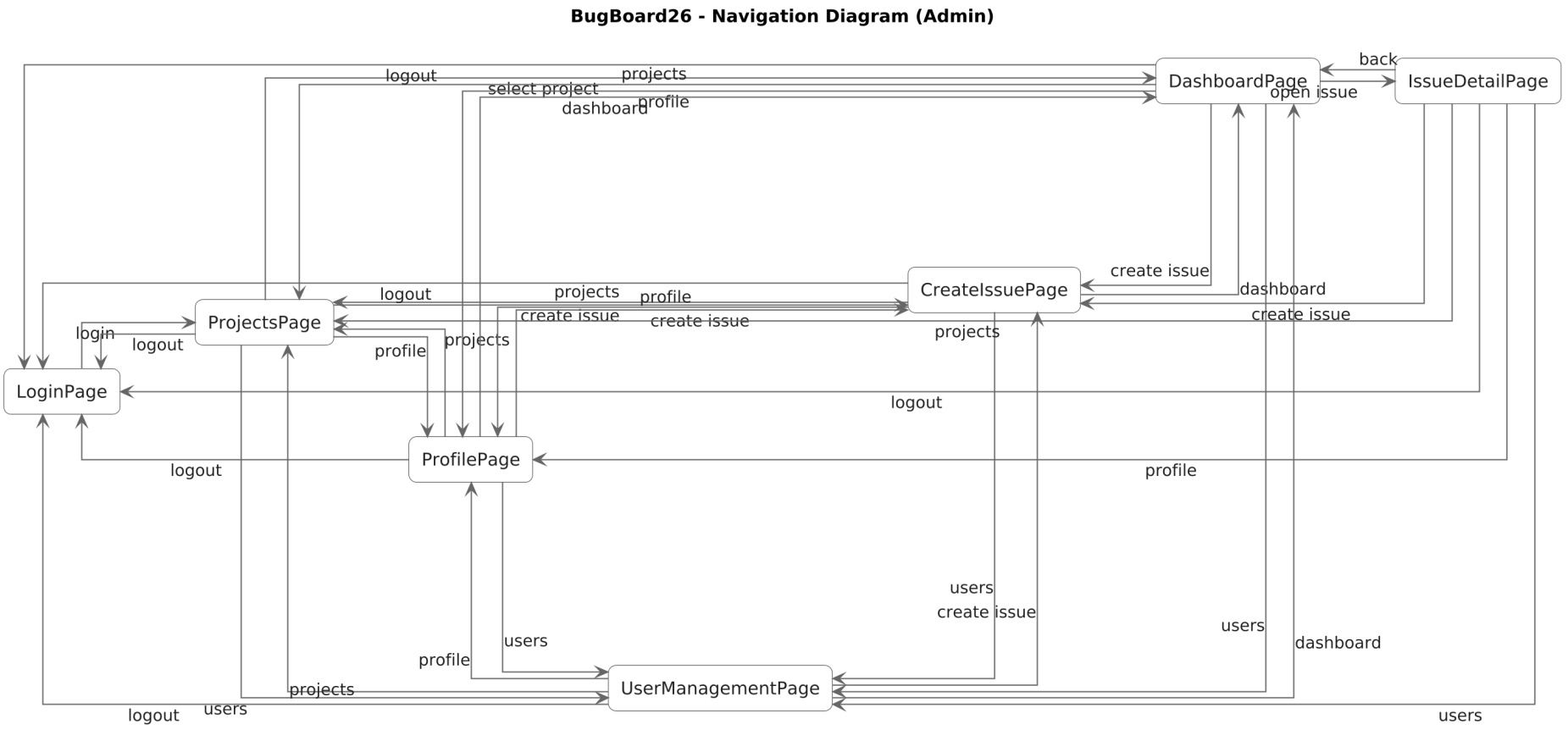


Figura 7.5: Schema originale della navigazione – Admin.

La navigazione tra le pagine è stata rappresentata tramite diagrammi separati rispetto al class diagram, così da non appesantire la vista strutturale del frontend. Il flusso principale prevede che, dopo l'autenticazione, l'utente venga indirizzato alla pagina dei progetti; una volta selezionato un progetto, viene aperta la dashboard delle issue relativa a quel progetto. Da qui è possibile navigare verso il dettaglio di una issue, la modale di creazione di una nuova issue, il placeholder del profilo e, nel caso dell'amministratore, anche la pagina di gestione degli utenti.

La navigazione è stata modellata distinguendo i due casi principali previsti dall'interfaccia:

- utente standard/developer, che può accedere a progetti, dashboard, creazione issue, dettaglio issue e placeholder del profilo;
- amministratore, che oltre a queste pagine può accedere anche alla gestione degli utenti.

Questa scelta ha permesso di rappresentare in modo più preciso le differenze di navigazione dovute al ruolo, mantenendo i diagrammi leggibili e coerenti con le voci effettivamente presenti nella sidebar e nella topbar dell'applicazione.

7.4 Sequenze di Design per gli Use Case

7.4.1 UC1 – Creazione issue

Frontend invia POST /issues; controller valida; createIssue apre la transazione, verifica progetto e assegnatario, salva TODO con reporter autenticato, aggiorna il progetto e produce l'eventuale notifica di assegnazione. Dopo commit restituisce il dettaglio; il frontend apre l'issue. Un errore nelle scritture o notifiche provoca rollback.

7.4.2 UC2 – Cambio stato

PATCH dello stato raggiunge changeStatus. requireIssue verifica accesso e blocca la riga; il servizio controlla la transizione, aggiorna issue/progetto e crea le notifiche. CLOSED genera RESOLUTION al reporter anche se attore o ex membro. Stesso stato: risposta idempotente; transizione illecita: conflitto; errore notifica: rollback.

7.4.3 UC3 – Commento

Il controller comments valida il testo e apre la transazione, verifica l'issue, salva il commento con autore server, aggiorna l'attività e chiama notifyParticipants. Dopo successo il frontend aggiunge il commento; in caso di errore conserva il testo.

7.4.4 UC4 – Creazione account e appartenenza

POST /users valida i campi, verifica email e salva l'account con hash BCrypt. La successiva richiesta di aggiunta al progetto è distinta: un errore di membership non elimina l'account già creato; l'interfaccia permette il recupero con Add Existing User.

Parte III

Documentazione del Processo di Sviluppo

Capitolo 8

File di build automatica

8.1 DockerFile (Backend)

Il Dockerfile presente usa node:24-alpine, installa le dipendenze del lockfile con npm ci –omit=dev e copia src, scripts e sql. L'esecuzione avviene come utente node; la porta esposta è 8080 e l'avvio usa src/server.js.

Fase	Istruzione / responsabilità
Base e directory	FROM node:24-alpine; WORKDIR /app.
Dipendenze	COPY package.json package-lock.json; RUN npm ci –omit=dev.
Sorgenti	Copia src, scripts e sql con proprietà node:node.
Esecuzione	USER node; EXPOSE 8080; CMD node src/server.js.

8.2 DockerFile (Frontend)

Nello snapshot non è presente un Dockerfile frontend. Il frontend viene installato con npm ci e avviato con Vite; npm run build produce dist/ e npm run preview ne permette la verifica locale. Un container per pubblicare la build è un eventuale completamento infrastrutturale, non un artefatto già fornito.

Riferimento ai sorgenti: backend/Dockerfile; frontend/package.json; infra/docker/docker-compose.yml.

8.3 Docker Compose

Il Compose contiene i servizi backend e postgres. PostgreSQL usa postgres:17-alpine e un volume persistente; il backend attende l'healthcheck DB. Le variabili derivano dalla configurazione d'esempio da adattare all'ambiente reale.

Servizio	Porta / configurazione	Persistenza
backend	8081 host → 8080 container; variabili DB, JWT e admin.	Codice nell'immagine; dati su PostgreSQL.

Servizio	Porta / configurazione	Persistenza
postgres	POSTGRES_PORT → 5432; porta prevista localmente 5433.	Volume postgres_data.
frontend	Servizio non presente; Vite locale su 5173.	Build frontend generata separatamente.

Inizializzazione e migrazione db:init esegue gli script 001 e 002 in transazione. Il server non applica automaticamente la migrazione; le note non confermano che sia stata applicata al DB di sviluppo. Prima di aggiornare un DB esistente, conservarne configurazione e backup. Non usare down -v sui dati da mantenere.

Riproducibilità dei controlli backend/test-docker.ps1 crea un PostgreSQL temporaneo con nome casuale e porta disponibile, esegue i test e rimuove soltanto il container creato. Lint e build frontend sono script separati. Gli esiti precedenti sono dichiarati nelle note, non verificati nuovamente in questa revisione.

Capitolo 9

Strumento di Versioning

Le note del pacchetto descrivono un progetto versionato con Git. Lo snapshot contiene frontend e backend nello stesso albero codice/, insieme a SQL, test, infrastruttura e documentazione. Il repository .git è escluso; non è disponibile una cronologia da analizzare direttamente.

Informazione	Evidenza disponibile
Commit dello snapshot	55fa6f9, dichiarato nel LEGGIMI: Align issue priority, TODO status and resolution notifications.
Migrazione da Spring	Le note citano 3144a8d; l'implementazione corrente è Express/Sequelize.
Cartella pulita	Dichiarazione del preparatore del pacchetto; non verificabile senza repository.
Contributori e andamento	Non ricostruibili dall'archivio privo di .git.
URL repository	Non documentato in modo verificabile dai materiali esaminati: non viene inventato.

Evidenze per la consegna Dal repository originale vanno esportati log dei commit, autori, date e statistiche dei contributi, specificando branch e commit analizzati. La relazione deve distinguere i commit di codice da quelli di sola documentazione e spiegare le principali tappe dello sviluppo.

Riferimento ai sorgenti: 00-LEGGIMI.md; 01-STATO-PROGETTO.md. Il riferimento immobiliare fornisce la struttura del capitolo, non un repository utilizzabile per BugBoard26.

Capitolo 10

Qualità del codice

Nello snapshot sono presenti ESLint per il frontend e una suite API backend. Non è incluso un report SonarQube o equivalente; non è quindi documentabile un Quality Gate superato o una misura di duplicazione, vulnerabilità e code smell.

Controllo	Stato documentabile	Evidenza ancora necessaria
Lint frontend	Script presente; esito precedente dichiarato.	Log riproducibile con commit, data e versione strumenti.
Build frontend	Script Vite presente; esito precedente dichiarato.	Log di build della versione di consegna.
Test backend	40 casi presenti; esito precedente dichiarato.	Esecuzione sullo snapshot con ambiente e report.
Analisi statica backend	Report non allegato.	Report SonarQube o equivalente con regole e perimetro.
Coverage	Report strumentale non allegato.	Misure per file, righe e rami con configurazione di inclusioni/esclusioni.

La revisione documentale non ha eseguito nuove analisi del prodotto. I capitoli seguenti definiscono cosa misurare e come riportarlo; le celle prive di risultato non indicano zero difetti o zero copertura.

10.1 Grafico di Duplicazione del Codice

Il grafico non è prodotto perché manca una misura strumentale dello snapshot. La sezione è mantenuta per corrispondenza con il riferimento e definisce il contenuto del report da allegare.

Metrica	Modalità di raccolta	Stato
Righe/blocchi duplicati	Scanner configurato sul codice backend, escludendo dipendenze e output.	Non rilevato
Duplicazione percentuale	Stesso scanner e stesso perimetro del report.	Non rilevato

Metrica	Modalità di raccolta	Stato
Trend	Confronto fra commit identificati; stessa configurazione.	Non disponibile

10.2 Grafico delle Issues

Separare bug, vulnerabilità e code smell, registrando severità, stato, regola e decisione di risoluzione. Non vengono riportati i numeri del progetto immobiliare: si riferiscono a un altro codice e a un altro framework.

10.3 Grafico della Coverage dei Test

Mostrare linee e rami coperti sul backend con file esclusi espliciti. Il numero di test non è una percentuale di copertura. Non è corretto attribuire 100% di node/branch coverage o un rapporto di efficacia senza i relativi report e una definizione del denominatore.

Esito attuale di 10.1-10.3: misure mancanti. È necessario eseguire gli strumenti sulla versione destinata alla consegna, conservare il report e poi inserire i grafici reali.

Parte IV

Testing e Valutazione dell'Usabilità

Capitolo 11

Unit Testing

La suite fornita contiene 40 test node:test basati su richieste HTTP e PostgreSQL isolato. La loro natura è prevalentemente integrativa/API. Questo capitolo distingue tali test dalle specifiche di unit test proposte per quattro funzioni non banali con almeno due parametri.

Sezione	Funzione selezionata	Ragione
11.1	createIssue(user, input)	Accesso, assegnazione e scritture atomiche.
11.2	changeStatus(user, issueId, status)	Transizioni, idempotenza e notifiche.
11.3	requireAssignee(projectId, userId, tx)	Lookup e vincolo di appartenenza.
11.4	notify(issue, actor, message, ids, tx)	Deduplicazione e selezione destinatari.

Stato delle specifiche di test
Le sezioni 11.1-11.4 progettano i test, ma non dichiarano una nuova suite unitaria implementata o eseguita. I test API già presenti non sono riclassificati come unit test. L'accettazione del framework richiesto dalla traccia va verificata sul testo ufficiale.

Ogni specifica definisce unità sotto test, dipendenze da sostituire, classi di equivalenza, casi limite, oracolo e obiettivo di copertura. Risultati osservati e copertura misurata saranno aggiunti soltanto dopo l'esecuzione.

11.1 Test di createIssue (issues service)

Unità: createIssue(user, input)

Isolamento: Sostituire transazione Sequelize, controlli accesso, modello Issue, touchProject, notifyAssignment e loadIssueResponse. Verificare dati inviati e uso della stessa transaction.

Caso / classe	Stimolo e stato simulato	Oracolo atteso
C1 - valido	Membro autorizzato; campi validi; assegnatario membro.	Issue con reporterId utente e status TODO; touch e notifica nella transaction.
C2 - facoltativi	priority=null e assigneeId=null.	Campi null conservati; nessun default di priorità inventato.
C3 - accesso	requireProject rifiuta.	Errore propagato; nessuna Issue.create.
C4 - assegnatario	requireAssignee rifiuta.	Nessuna scrittura issue; transazione fallita.
C5 - notifica	notifyAssignment fallisce dopo creazione.	Rifiuto della transaction; nessun esito di successo.
C6 - identità	Input con reporterId/stato estranei al server.	Il servizio sovrascrive reporterId e status; validazione campi nel controller testata separatamente.

Criteri white-box e casi limite Coprire il percorso nominale e le interruzioni nelle dipendenze. Il rollback reale richiede anche un test di integrazione sul DB: un mock della transaction da solo non prova che PostgreSQL conservi l'atomicità.

Risultato osservato: non eseguito. Copertura: non misurata. I casi sono specifiche da trasformare in test unitari con asserzioni e dipendenze isolate.

11.2 Test di changeStatus (issues service)

Unità: changeStatus(user, issueId, status)

Isolamento: Simulare requireIssue, issue.update, transazione, touchProject, funzioni notify e loadIssueResponse. Stabilire stato, reporter, assegnatario e attore.

Caso / classe	Stimolo e stato simulato	Oracolo atteso
S1 - idempotenza	Nuovo stato uguale allo stato attuale.	Nessun update e nessuna nuova notifica.
S2 - avanzamento	TODO → IN_PROGRESS.	Update, touchProject e notifyParticipants.
S3 - chiusura	IN_PROGRESS → CLOSED, assignee diverso dal reporter.	notifyResolution e notifica attività prevista all'assegnatario.
S4 - coincidenza	Chiusura con reporter=assignee, anche attore.	Una risoluzione al reporter; destinatari attività senza duplicazione.
S5 - transizione	TODO → CLOSED o CLOSED → IN_PROGRESS.	ApiError 409; nessun aggiornamento.
S6 - errore	Accesso negato o notifyResolution rifiutata.	Errore propagato; operazione transazionale fallita.

Criteri white-box e casi limite Rami: stato invariato, transizione valida/non valida, CLOSED/non CLOSED e reporter=assignee. La concorrenza di due chiusure va verificata con test integrativi: non può essere provata con una singola istanza simulata.

Risultato osservato: non eseguito. Copertura: non misurata. I casi sono specifiche da trasformare in test unitari con asserzioni e dipendenze isolate.

11.3 Test di requireAssignee (access service)

Unità: requireAssignee(projectId, userId, transaction)

Isolamento: Sostituire User.findByPk e Membership.findOne; verificare projectId, userId e passaggio della transaction.

Caso / classe	Stimolo e stato simulato	Oracolo atteso
A1 - null	userId=null.	Ritorna null; nessun lookup.
A2 - inesistente	Account non trovato.	ApiError 404 User not found.
A3 - esterno	Account trovato, nessuna membership.	ApiError 400 ASSIGNEE_NOT_MEMBER.
A4 - membro	Account e membership presenti.	Ritorna l'account trovato.
A5 - ADMIN esterno	Account ADMIN senza membership.	Stesso errore A3: il ruolo non esenta il vincolo di assegnazione.
A6 - transaction	Stesso caso valido con transaction esplicita.	Entrambi i lookup ricevono la transaction.

Criteri white-box e casi limite Partizioni: null / account assente / account presente non membro / membro. Criterio: coprire tutti i ritorni e i throw espliciti. ID sintatticamente non validi sono responsabilità della validazione a monte e richiedono test dedicati di validation.id.

Risultato osservato: non eseguito. Copertura: non misurata. I casi sono specifiche da trasformare in test unitari con asserzioni e dipendenze isolate.

11.4 Test di notify (notifications service)

Unità: notify(issue, actor, message, recipientIds, transaction)

Isolamento: Sostituire User.findByPk, Membership.findOne e Notification.create. Usare destinatari duplicati, attore e account con diversi permessi.

Caso / classe	Stimolo e stato simulato	Oracolo atteso
N1 - precondizione	transaction assente.	Errore prima di qualsiasi scrittura.
N2 - lista vuota	Nessun destinatario valido o soltanto attore.	Nessuna Notification.create.

Caso / classe	Stimolo e stato simulato	Oracolo atteso
N3 - duplicati	Stesso ID in forme equivalenti e valori vuoti.	Una sola notifica per destinatario; valori vuoti esclusi.
N4 - accesso	Membro e ADMIN distinti dall'attore.	Notifica agli autorizzati; ADMIN non richiede membership.
N5 - esclusioni	Account inesistente oppure USER non membro.	Nessuna notifica per tali ID.
N6 - errore DB	Notification.create rifiuta.	Errore propagato al chiamante transazionale.

Criteri white-box e casi limite Coprire: transaction sì/no, lista vuota/non vuota, attore/destinatario, account esistente/assente e ADMIN/membro/esterno. La funzione tratta ACTIVITY; RESOLUTION segue notifyResolution e deve essere testata con il relativo contratto speciale.

Risultato osservato: non eseguito. Copertura: non misurata. I casi sono specifiche da trasformare in test unitari con asserzioni e dipendenze isolate.

11.5 Strategie adottate

11.5.1 Controller

I test presenti attraversano HTTP, middleware, controller e persistenza. Verificano status code, JSON, identità server, validazione e permessi. Per isolare un controller occorre invece simulare req/res e dipendenze, evitando di avviare il database. I controlli del backend restano necessari anche quando il frontend valida gli stessi campi.

11.5.2 Service

Le specifiche 11.1-11.4 combinano partizionamento degli input e analisi dei rami. Si controllano chiamate, argomenti, esiti e propagazione errori; gli stub devono preservare il contratto delle dipendenze. È necessario rendere ripetibili identità, date e transazioni simulate.

11.5.3 Model / persistenza

Il corrispondente della sezione Repository del riferimento è qui la persistenza Sequelize. Vincoli univoci, chiavi esterne, lock, migrazioni ripetibili e rollback devono essere provati su PostgreSQL isolato. Un mock ORM non è sufficiente per dimostrarli.

11.5.4 Security e Utils

La suite API include token mancanti, alterati e scaduti; account e ruoli vengono riletti dal DB. Ulteriori unit test possono isolare validatori e serializzazione. Per la password occorre distinguere caratteri e byte UTF-8; per gli ID BIGINT, range SQL e rappresentazione numerica JavaScript.

Framework attuale: node:test. Nessun uso di JUnit, MockMvc, JaCoCo o Mockito è dichiarato per BugBoard26. I risultati unitari e la coverage restano da produrre.

11.6 Test plan di una funzionalità: risoluzione issue

Elemento	Piano
Obiettivo	Verificare chiusura, notifica al reporter e conservazione dei permessi.
Ambiente	Backend dello snapshot, PostgreSQL isolato, frontend con API reali; registrare versioni e commit.
Dati	ADMIN A; reporter R; assegnatario S; esterno X; progetto con issue IN_PROGRESS. Account distinti salvo varianti esplicite.
Precondizioni	Migrazione 002 applicata; token validi; membership controllate; notifiche iniziali registrate.
Criterio di ingresso	Avvio riuscito e dati noti; nessuna dipendenza da database di sviluppo.
Criterio di uscita	Tutti i casi sotto eseguiti con risultato osservato e anomalie registrate. Nessuna percentuale dichiarata a priori.

Caso	Azione	Atteso
TP1	S chiude IN_PROGRESS.	CLOSED persistito, notifica RESOLUTION a R.
TP2	Ripetere chiusura; anche concorrente.	Una sola risoluzione per il cambio stato.
TP3	R coincide con attore o ex membro.	Notifica personale leggibile; nessun accesso riacquisito alla issue.
TP4	X tenta chiusura; transizione TODO → CLOSED.	Accesso negato / conflitto; nessuna modifica.
TP5	Errore notifica e successivo riavvio.	Rollback sull'errore; dati riusciti conservati dopo riavvio.

Esecuzione del piano: da svolgere. Per ogni caso registrare data, input, esito reale, evidenza e identificativo dell'eventuale difetto.

Capitolo 12

Valutazione dell'Usabilità

12.1 Expert Review

12.1.1 Obiettivo del Test

Valutare la comprensibilità dei flussi di segnalazione, collaborazione e amministrazione di BugBoard26 secondo le otto regole d'oro di Shneiderman. La review deve includere navigazione, feedback, recupero errori e coerenza dei permessi visibili.

12.1.2 Istruzioni per l'Uso della Checklist

Ogni valutatore esegue gli stessi compiti con account e dati predisposti. Per ciascuna regola assegna 1-5, oppure "non valutabile" con motivo, annotando schermata, evidenza e severità. I punteggi sono aggregati solo sulle risposte valide, riportando anche numerosità e dispersione.

Punteggio	Interpretazione
1-2	Problema rilevante; documentare l'impedimento e proporre correzione.
3	Supporto parziale o comportamento da migliorare.
4-5	Supporto adeguato secondo l'osservazione del valutatore.
Non valutabile	Dati o funzione non disponibili: escludere dalla media, dichiarando il motivo.

12.1.3 Risultati

Non sono disponibili schede compilate o valutatori documentati per BugBoard26. I risultati dell'altro progetto non vengono trasferiti. Non si calcolano medie né si dichiara un esito positivo prima di svolgere la review.

Checklist dell'Expert Review

Regola	Cosa osservare su BugBoard26	Stato
Coerenza	Stessi nomi TODO/IN_PROGRESS/CLOSED e priorità null nelle viste.	Da valutare
Scorciatoie	Accesso rapido a progetto, issue e azioni frequenti senza percorsi ridondanti.	Da valutare
Feedback informativo	Attesa, successo ed errore percepibili, compresi invii falliti.	Da valutare
Chiusura del dialogo	Creazione conclusa nel dettaglio; commento aggiunto visibile.	Da valutare
Prevenzione errori	Campi, stati disponibili e conferma rimozione membri.	Da valutare
Annullamento/recu	Cancel, testo conservato dopo errore; limite di chiusura irreversibile esplicito.	Da valutare
Controllo utente	Permessi e azioni disponibili comprensibili; nessuna falsa funzione completa.	Da valutare
Memoria a breve termine	Contesto progetto, reporter, assegnatario e filtri visibili.	Da valutare

Per ogni riga conservare punteggio, nota, schermata e raccomandazione di ciascun valutatore. Accessibilità da tastiera, ordine di focus e contrasto vanno aggiunti come controlli tecnici dedicati, senza dedurli automaticamente da una media della checklist.

12.2 Relazione Sperimentale

12.2.1 Descrizione dei Partecipanti

Nessun campione è documentato per BugBoard26. Il protocollo propone utenti reali o potenziali con esperienza digitale diversa, includendo membri di team e persone che svolgano il ruolo amministrativo. Registrare identificativi pseudonimi, esperienza con issue tracker e familiarità con il prodotto; non inventare partecipanti o età.

12.2.2 Procedura Sperimentale

1. Presentare obiettivo, modalità di osservazione e compiti; usare un ambiente di prova con dati fittizi.

2. Assegnare un account USER o ADMIN e verificare dati iniziali identici per compiti comparabili.

3. Osservare i compiti senza indicare i pulsanti; annotare richieste di aiuto e deviazioni.

4. Registrare completamento, tempi, errori e commenti; raccogliere la valutazione post-task.

5. Ripristinare i dati tra sessioni e analizzare criticità per task e profilo.

Task	Consegna all'utente
U1	Apri un progetto e segnala un BUG senza priorità.

Task	Consegna all'utente
U2	Trova una issue usando filtri combinati e inserisci un commento.
U3	Porta l'issue a CLOSED e verifica la notifica con l'account reporter.
A1	Crea un account, aggiungilo al progetto e poi rimuovilo con conferma.

12.2.3 Dati Raccolti

Lo schema di raccolta è definito; non sono ancora presenti osservazioni sperimentali. Ogni riga dovrà rappresentare un task di un partecipante, non un valore attribuito a posteriori.

Variabile	Definizione operativa
Partecipante / ruolo	Codice pseudonimo e profilo USER/ADMIN usato per la prova.
Task / esito	Completato senza aiuto, con aiuto oppure non completato.
Tempo	Secondi dall'avvio del compito al criterio di completamento definito.
Errori	Numero e descrizione di azioni errate, blocchi e recuperi.
Aiuto	Richieste spontanee o interventi del moderatore.
Soddisfazione	Valutazione 1-5 raccolta dopo ciascun task.
Note qualitative	Aspettative, esitazioni e commenti, collegati a una schermata.

12.2.4 Valutazione post-task

Questionario proposto: "È stato facile completare il compito?", "I messaggi erano chiari?", "Ho capito l'esito dell'azione?" e "Mi sono sentito in controllo?". Scala da 1 (per nulla) a 5 (del tutto), più una domanda aperta sui problemi incontrati. Questo questionario non è presentato come una scala standardizzata o validata.

Al momento: nessuna risposta raccolta, nessun tempo misurato, nessun tasso di successo calcolabile. L'assenza di dati non corrisponde a un esito negativo o a zero errori.

12.2.5 Conclusioni

Il documento ora copre le quattro parti e i dodici capitoli della struttura di riferimento, adattandoli a BugBoard26. Requisiti, architettura e diagrammi sono ricavati dallo snapshot; build e test sono descritti senza attribuire risultati non disponibili.

Attività da completare prima della consegna

Ambito	Evidenza da aggiungere
Requisiti	Confronto con traccia ufficiale e chiarimento accessi per membership.
Ambiente	Stato migrazione DB, avvio e prova browser ADMIN/USER con persistenza.

Ambito	Evidenza da aggiungere
Versionamento	URL reale, log, commit e contributori dal repository originale.
Qualità	Report backend, duplicazione/issues/coverage e relativi grafici.
Unit testing	Implementazione ed esecuzione delle specifiche 11.1-11.4 con framework accettato; risultati e misure.
Usabilità	Expert review, partecipanti effettivi, osservazioni, questionario e analisi.
Presentazione	Preparazione e prova della demo e della presentazione tecnica.

Le funzioni extra non implementate, come upload immagini, storico, ricerca o eliminazione progetti, non diventano obblighi soltanto perché presenti in un esempio. La decisione sul perimetro finale deve risalire alla traccia del corso.

Fonti: sorgenti e note di BugBoard26-per-ChatGPT-2026-10-21.zip; documento BugBoard26 (4).pdf; struttura di Documentazione_IngSw (4).pdf. Il marchio UniNa in copertina proviene dal materiale istituzionale del documento di riferimento. Nessun dato sperimentale del progetto DietiEstates25 è riutilizzato per BugBoard26.